Computational Thinking and Artificial Intelligence

08. Neural Network, BackProp

Kyoungwon Seo
Dept. of Applied Artificial Intelligence
kwseo@seoultech.ac.kr

Computational Thinking & Artificial Intelligence

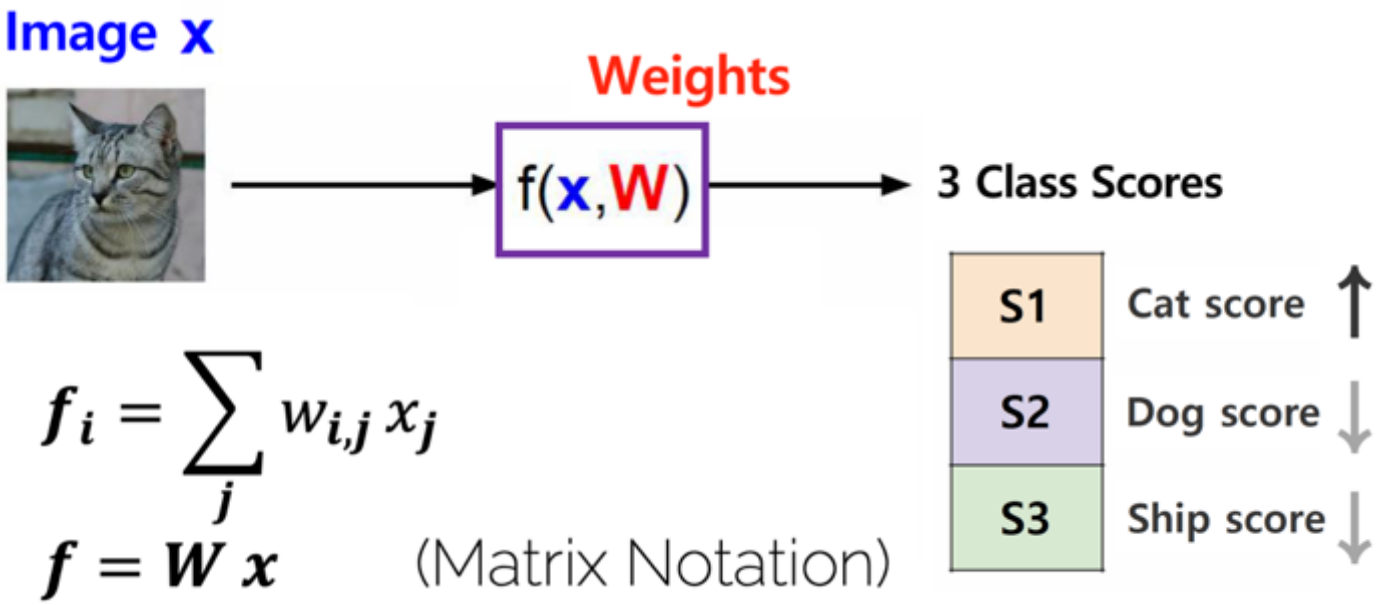
Lecture 08, 2025-Fall

1. Neural Network
2. Deeper Learning
3. Deep Learning

What is Artificial Neural Network?

What is Artificial Neural Network?

- **Linear Score Functions**
  - Linear score function as seen in linear regression



What is Artificial Neural Network?

- **Can we make linear score functions better?**
  - Multiply with another weight matrix W_2

$$\hat{f} = W_2 \cdot f$$
$$\hat{f} = W_2 \cdot W \cdot x$$

What is Artificial Neural Network?

- **Can we make linear score functions better?**
  - Multiply with another weight matrix W_2

$$\hat{f} = W_2 \cdot f$$
$$\hat{f} = W_2 \cdot W \cdot x$$

  - Operation is still linear

$$\widehat{W} = W_2 \cdot W$$
$$\hat{f} = \widehat{W}\, x$$

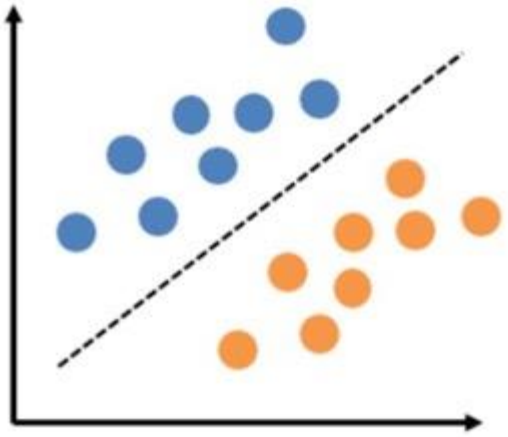
  - Solution → add ***non-linearity***!

What is Artificial Neural Network?

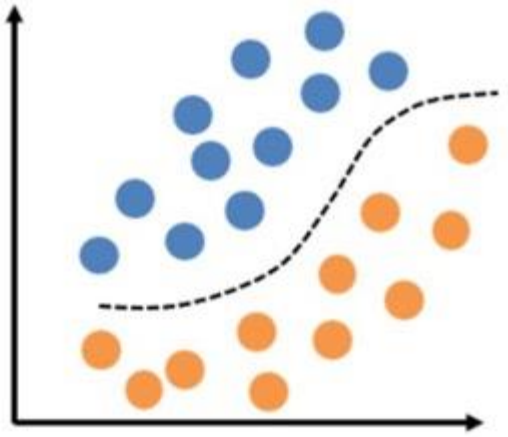
- **Linear score function:** $f = Wx$

- **Neural network is a nesting of 'functions'**
  - 2-layers: $f = W_2 \max(0, W_1 x)$
  - 3-layers: $f = W_3 \max(0, W_2 \max(0, W_1 x))$
  - 4-layers: $f = W_4 \tanh(W_3, \max(0, W_2 \max(0, W_1 x)))$
  - 5-layers: $f = W_5 \sigma(W_4 \tanh(W_3, \max(0, W_2 \max(0, W_1 x))))$
  - ... Up to hundreds of layers

Linear score function

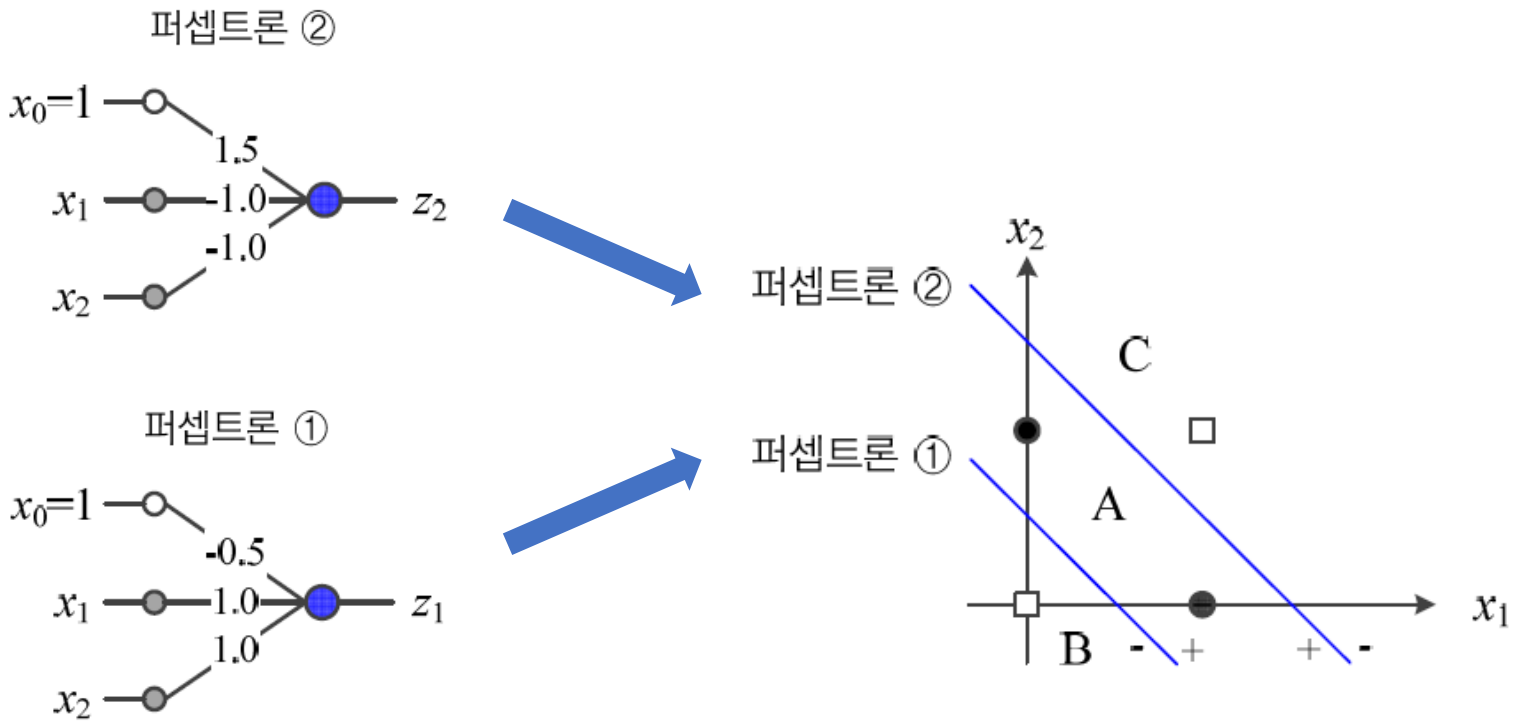


Neural network



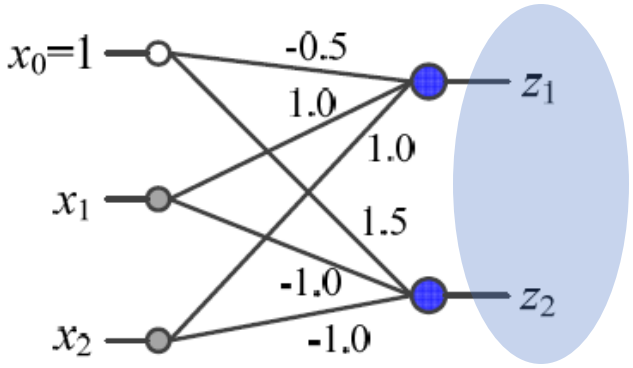
What is Artificial Neural Network?

- **Power of the multi-layer perceptron (Solving the XOR problem)**
  - If Perceptron① and Perceptron② are both +1 it is ●class, otherwise □class



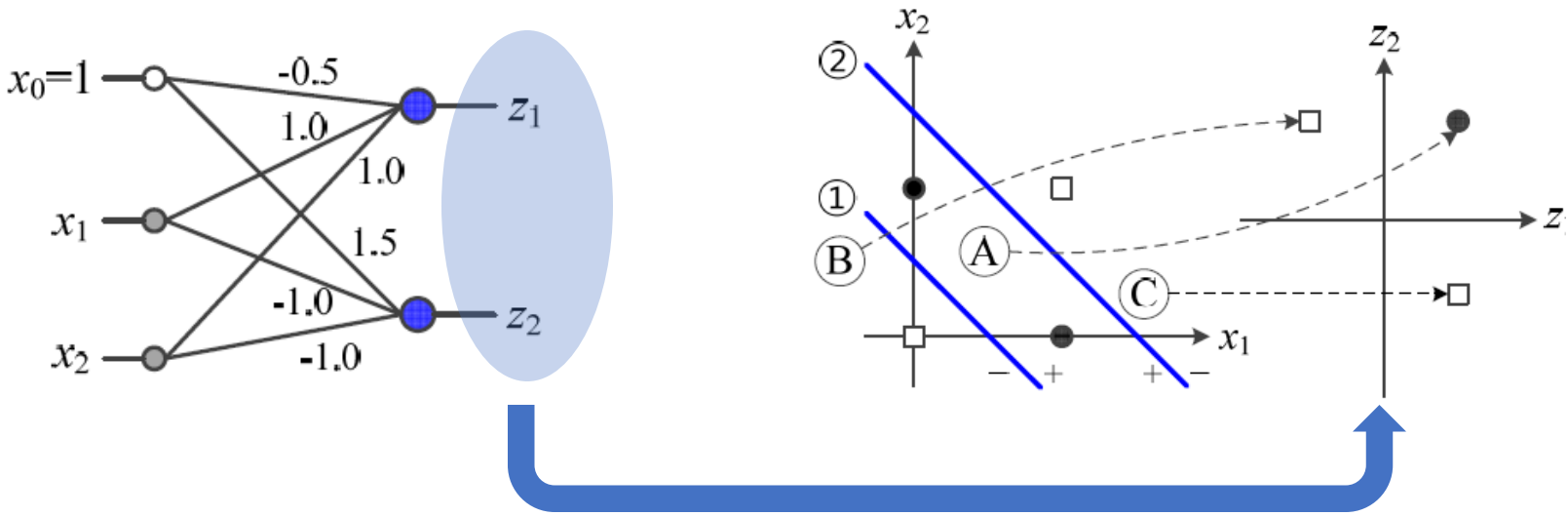
What is Artificial Neural Network?

- **Power of the multi-layer perceptron (Solving the XOR problem)**
  - If you combine two Perceptrons in parallel,
    Transform the original space $\mathbf{x} = (x_1, x_2)^{\mathrm{T}}$ → **New feature space** $\mathbf{z} = (z_1, z_2)^{\mathrm{T}}$
  - The XOR problem is now **linearly separable** in the new feature space $\mathbf{z}$!



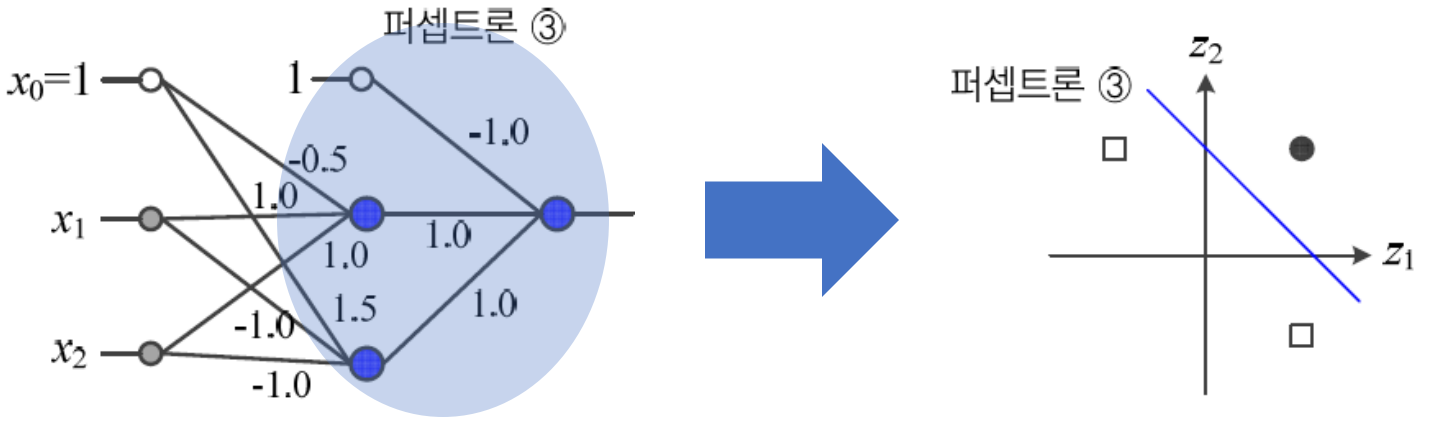
What is Artificial Neural Network?

- **Power of the multi-layer perceptron (Solving the XOR problem)**
  - If you combine two Perceptrons in parallel,
    Transform the original space $\mathbf{x} = (x_1, x_2)^{\mathrm{T}}$ → **New feature space** $\mathbf{z} = (z_1, z_2)^{\mathrm{T}}$
  - The XOR problem is now **linearly separable** in the new feature space $\mathbf{z}$!



What is Artificial Neural Network?

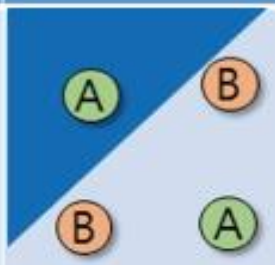
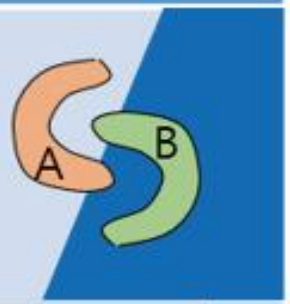
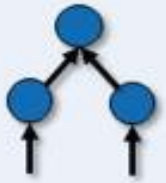
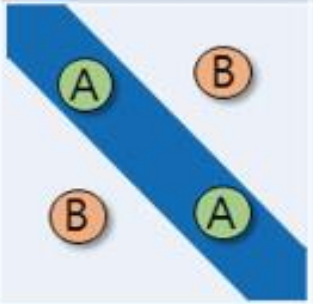
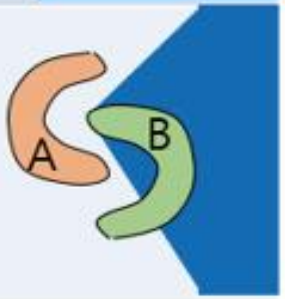
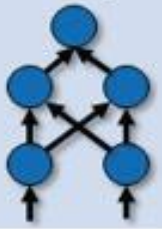
- **Power of the multi-layer perceptron (Solving the XOR problem)**
  - By sequentially combining Perceptron③, which performs linear separation in the new feature space $\mathbf{z}$, it becomes a ***multi-layer perceptron***



  - In general, combining *p* Perceptrons converts them into a *p*-dim space

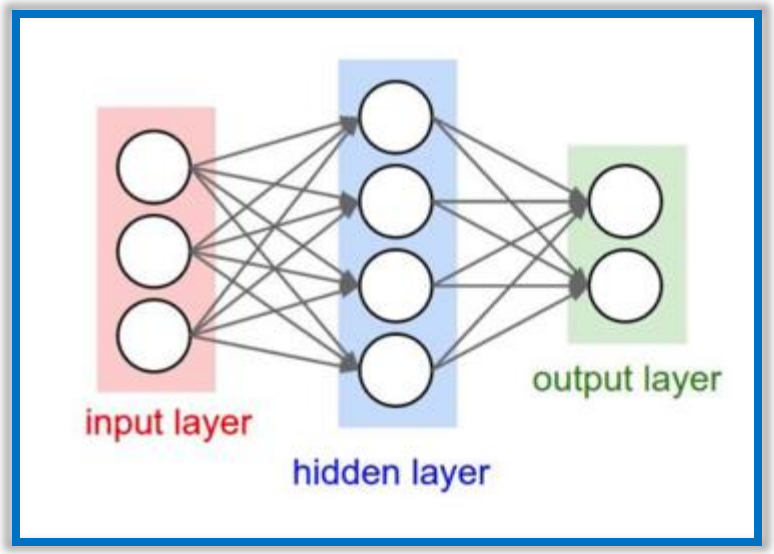
What is Artificial Neural Network?

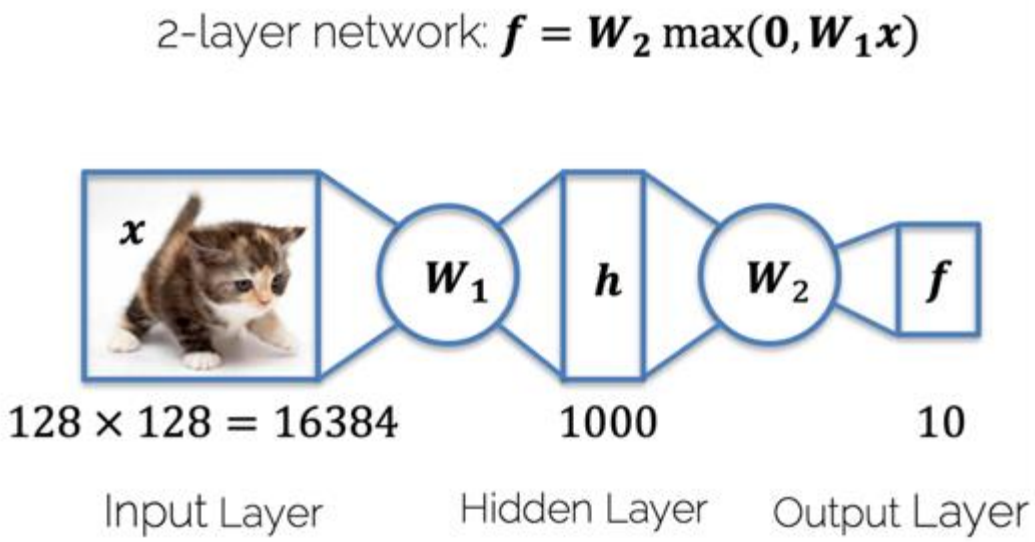
- **Power of the multi-layer perceptron**

Structure	Regions	XOR	Meshed Regions
Single layer 	Halfplane bounded by hyperplane		
Two layers 	Convex Open or closed regions		
Three layers 	Arbitrary (limited by # of nodes)		

What is Artificial Neural Network?

- **Structure of neural network**

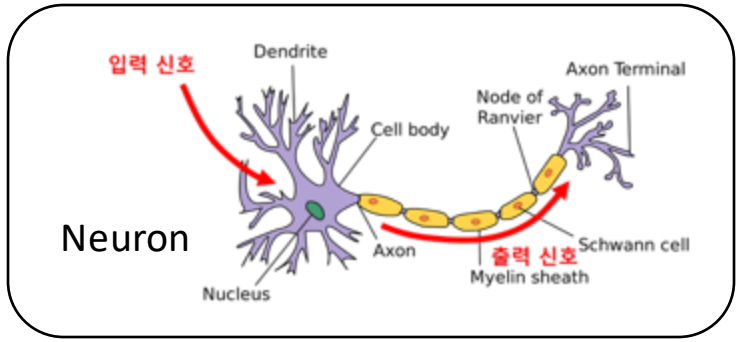


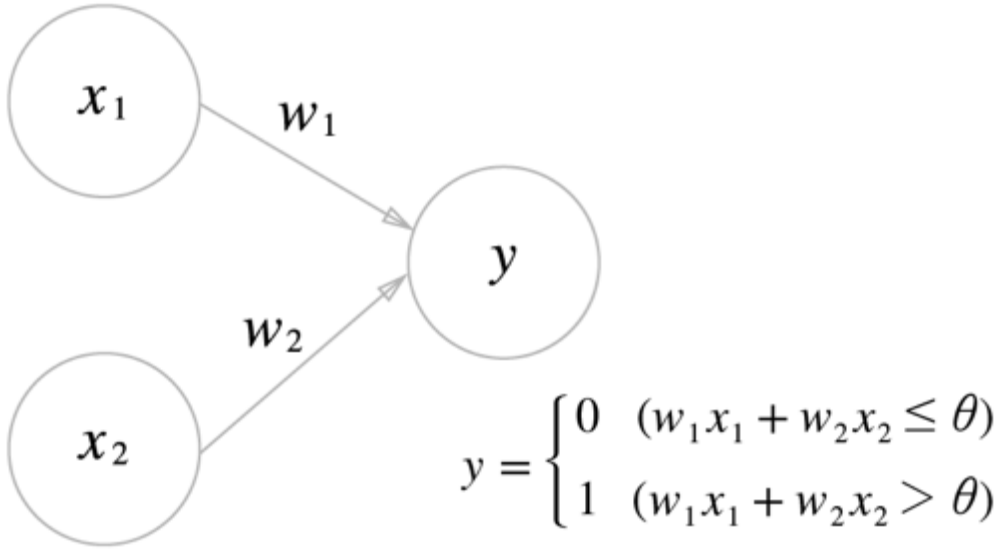


- *Easy to understand* complex structure of ANN
- *Simplify* complex mathematical formulas

What is Artificial Neural Network?

- **Structure of neural network: Perceptron**





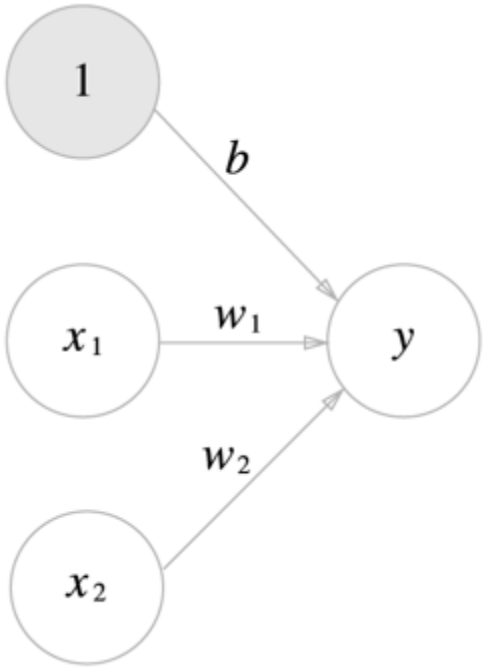
– An algorithm for learning a binary classifier called ***a threshold function***: a function that maps its *input x* to an *output y*

– There are *weight w* (a real-value vector) and *threshold* $\boldsymbol{\theta}$

– The circles are called *neuron* or *node*

What is Artificial Neural Network?

- **Structure of neural network: Bias**



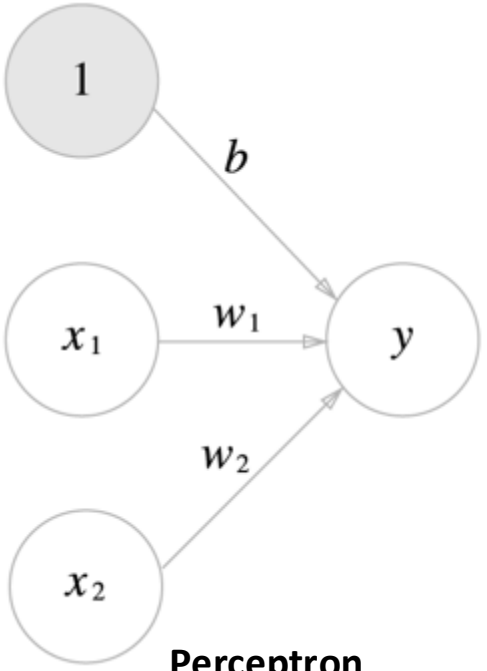
$$y = \begin{cases} 0 & (w_1 x_1 + w_2 x_2 \leq \theta) \\ 1 & (w_1 x_1 + w_2 x_2 > \theta) \end{cases}$$

$$y = \begin{cases} 0 & (b + w_1 x_1 + w_2 x_2 \leq 0) \\ 1 & (b + w_1 x_1 + w_2 x_2 > 0) \end{cases}$$

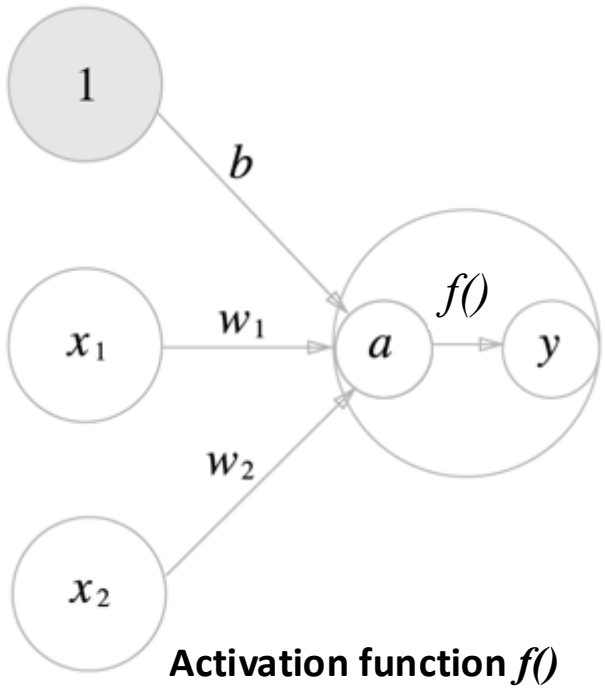
- Binary classification centered on 0 by replacing θ(theta) with *-b(bias)*

What is Artificial Neural Network?

- **Structure of neural network: Activation function**



Perceptron



Activation function *f()*

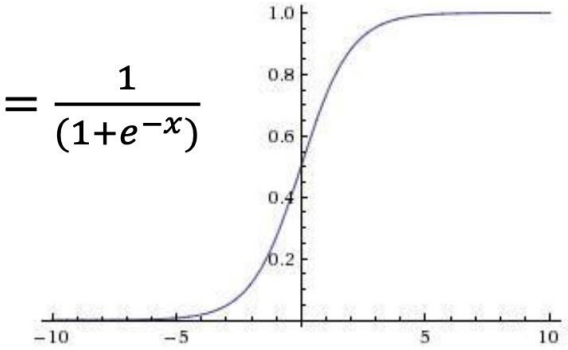
– A function that converts the sum of the input signal into a ***non-linear*** *output signal*

– Using a linear activation function doesn't give you the benefit of having multiple layers

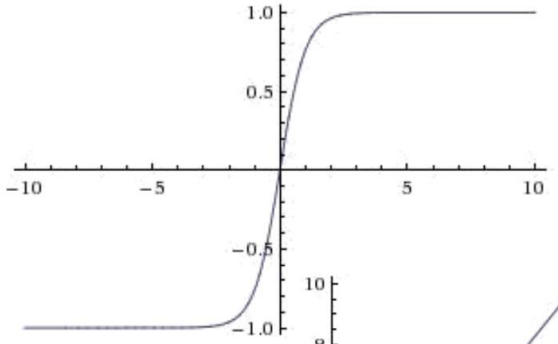
What is Artificial Neural Network?

- **Structure of neural network: Activation function**

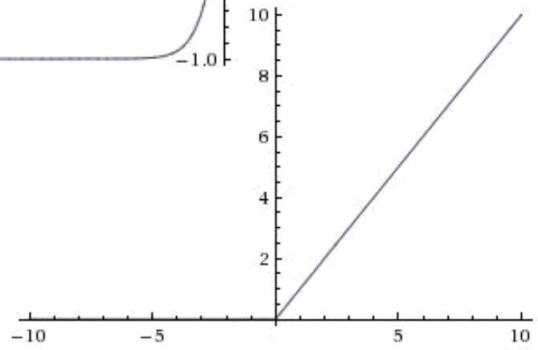
Sigmoid: $\sigma(x) = \frac{1}{(1+e^{-x})}$



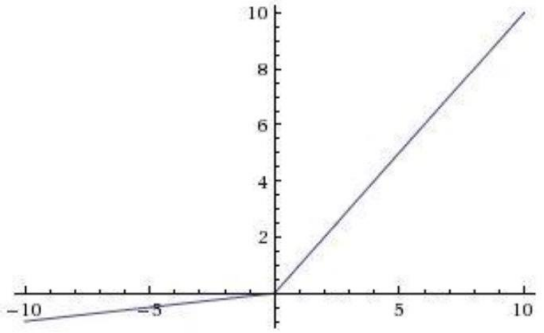
tanh: $\tanh(x)$



ReLU: $\max(0, x)$



Leaky ReLU: $\max(0.1x, x)$



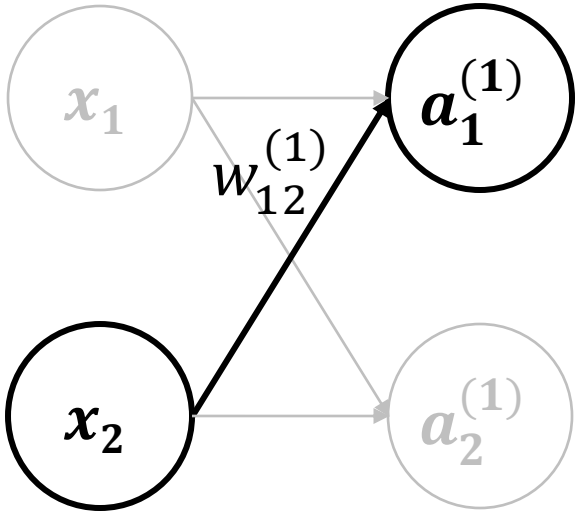
Parametric ReLU: $\max(\alpha x, x)$

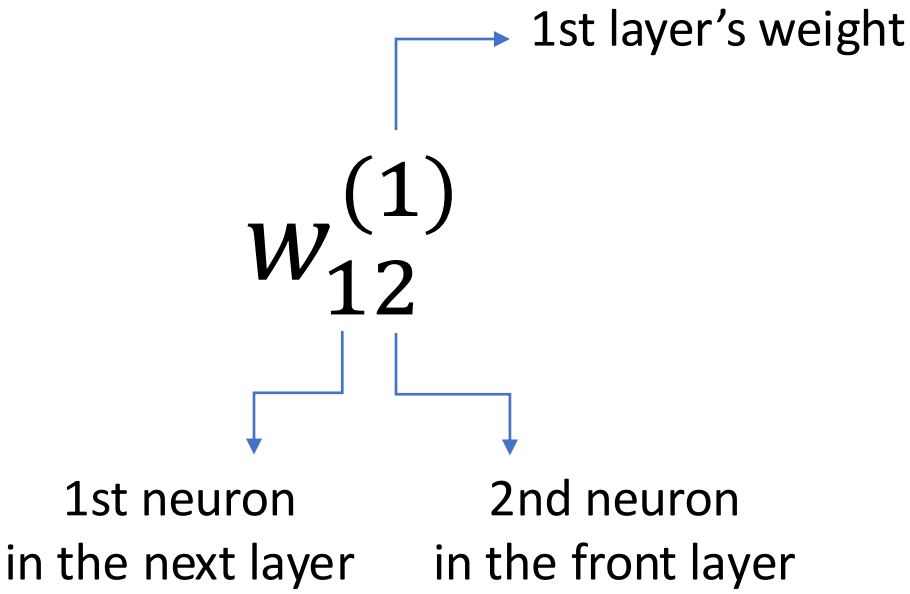
Maxout $\max(w_1^T x + b_1, w_2^T x + b_2)$

ELU $f(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha(e^x - 1) & \text{if } x \leq 0 \end{cases}$

What is Artificial Neural Network?

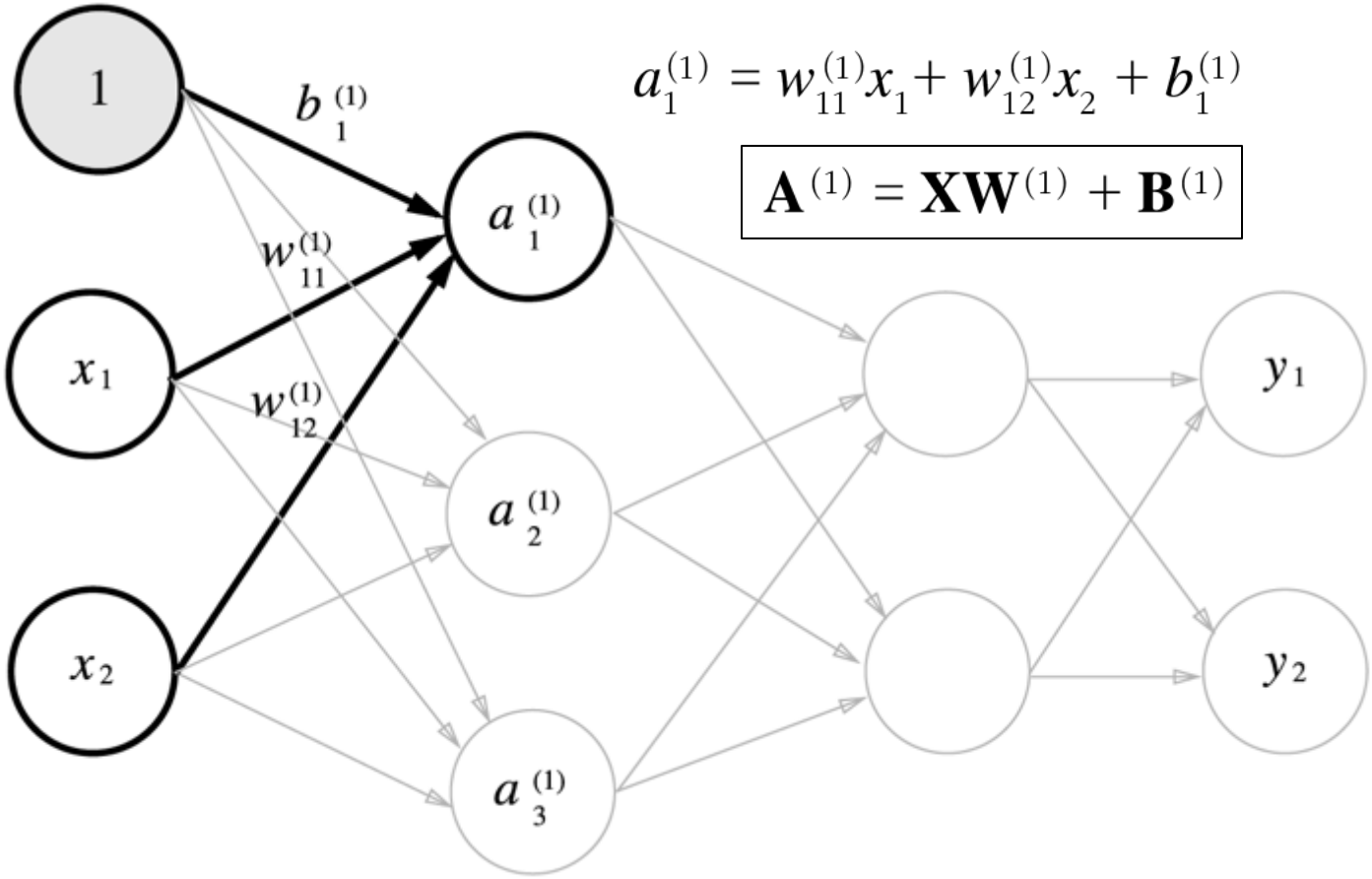
- **Structure of neural network: Notations**





What is Artificial Neural Network?

- **Structure of neural network: Feedforward**

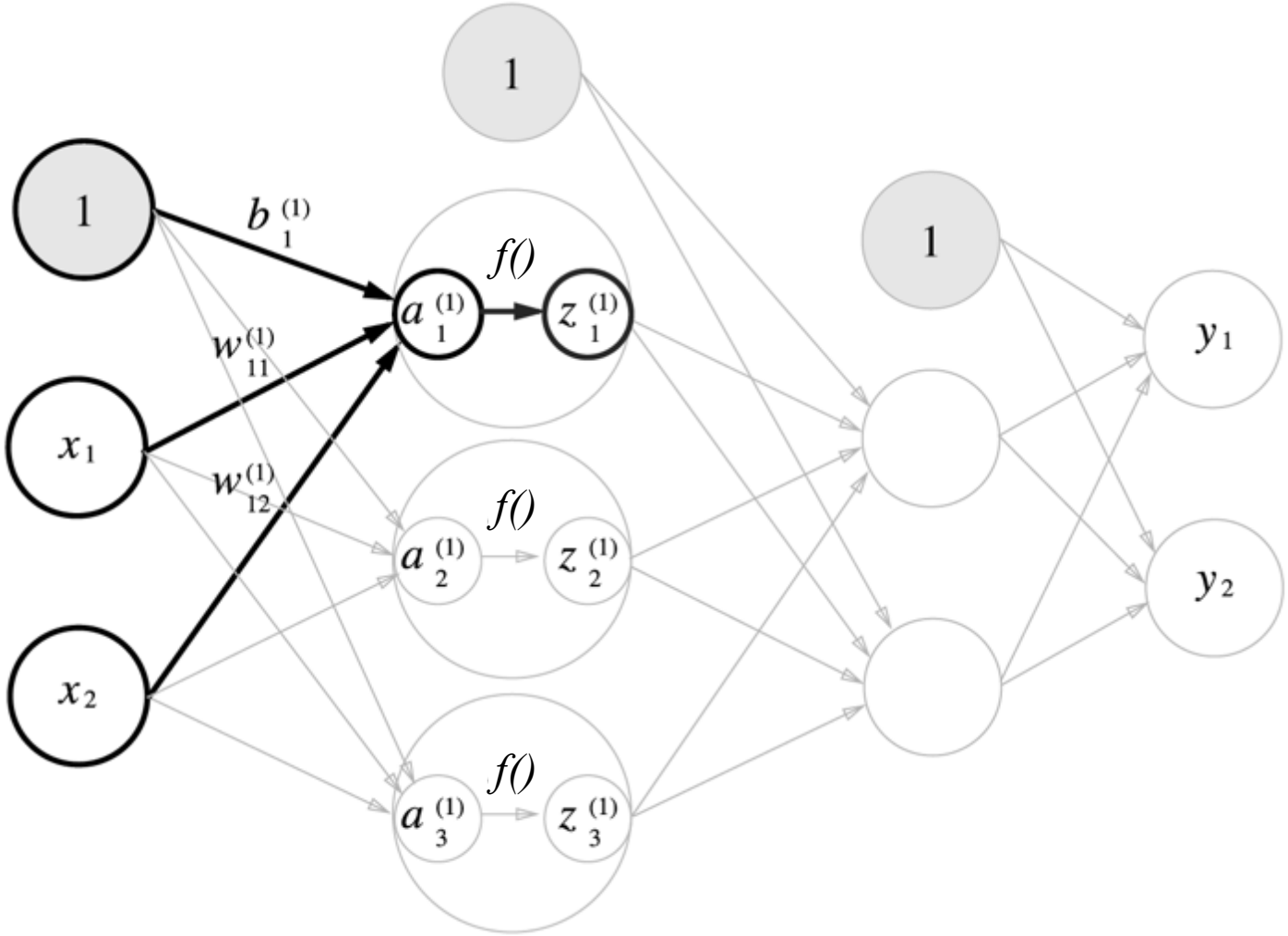


$$a_1^{(1)} = w_{11}^{(1)}x_1 + w_{12}^{(1)}x_2 + b_1^{(1)}$$

$$\mathbf{A}^{(1)} = \mathbf{X}\mathbf{W}^{(1)} + \mathbf{B}^{(1)}$$

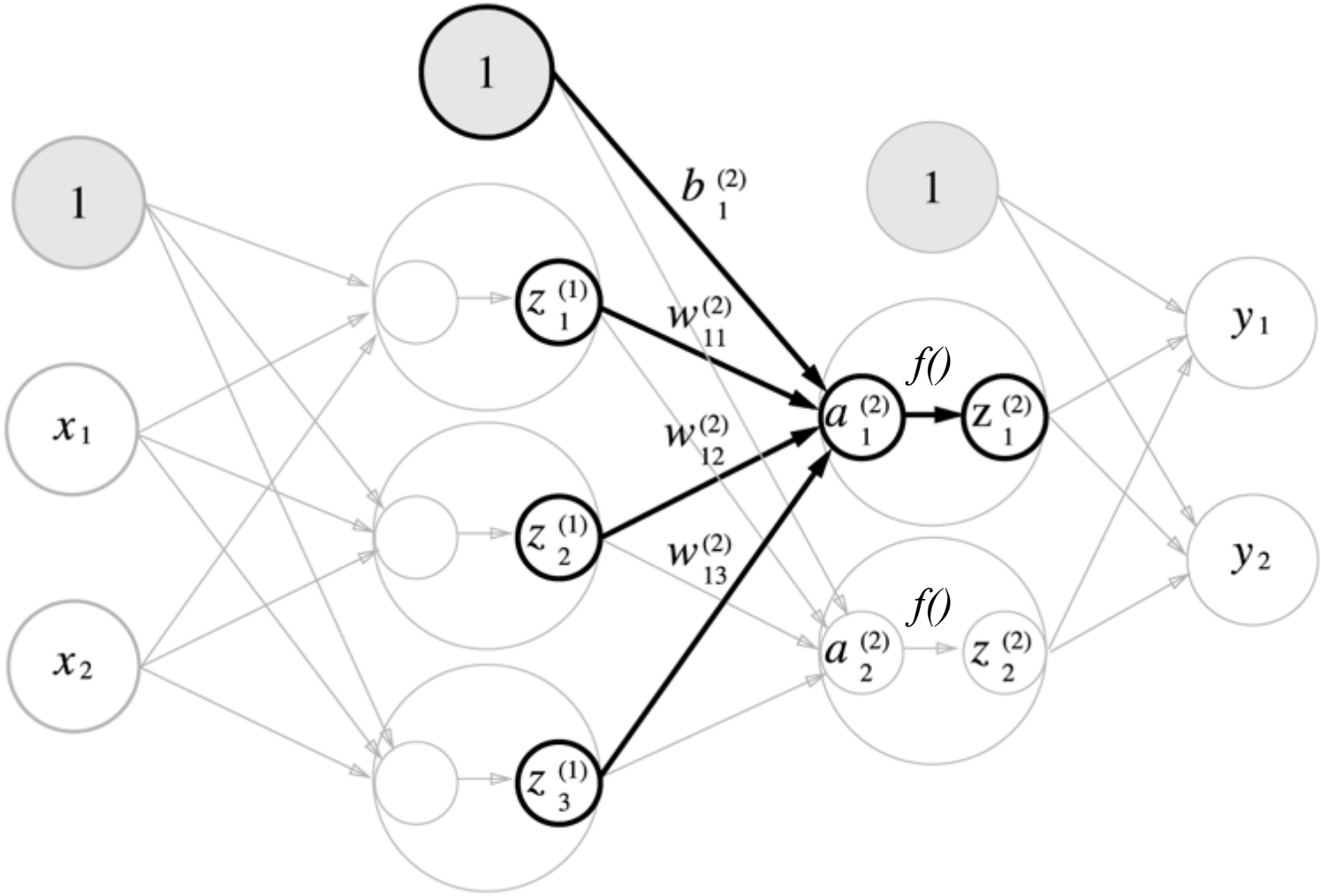
What is Artificial Neural Network?

- **Structure of neural network: Feedforward**



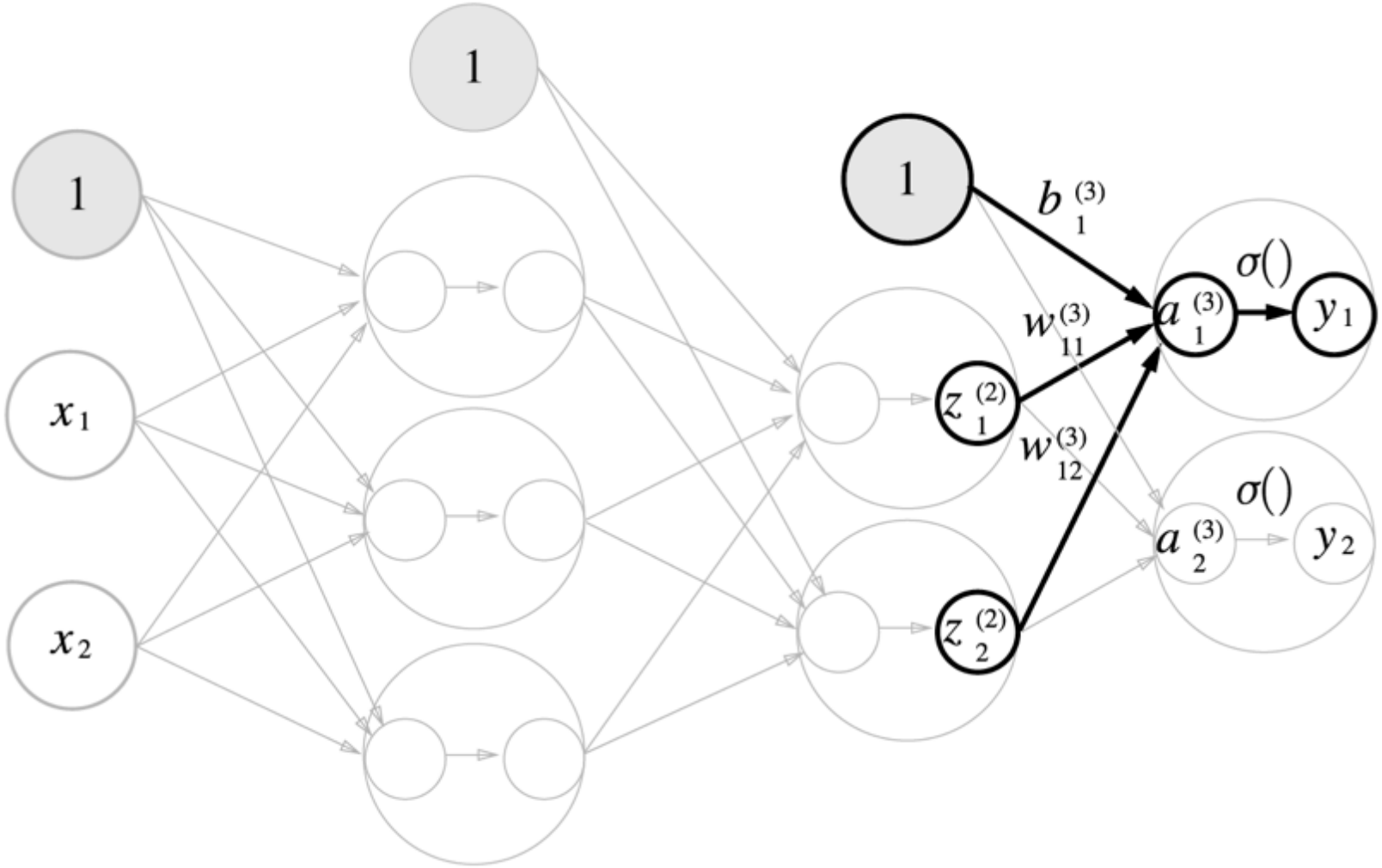
What is Artificial Neural Network?

- **Structure of neural network: Feedforward**



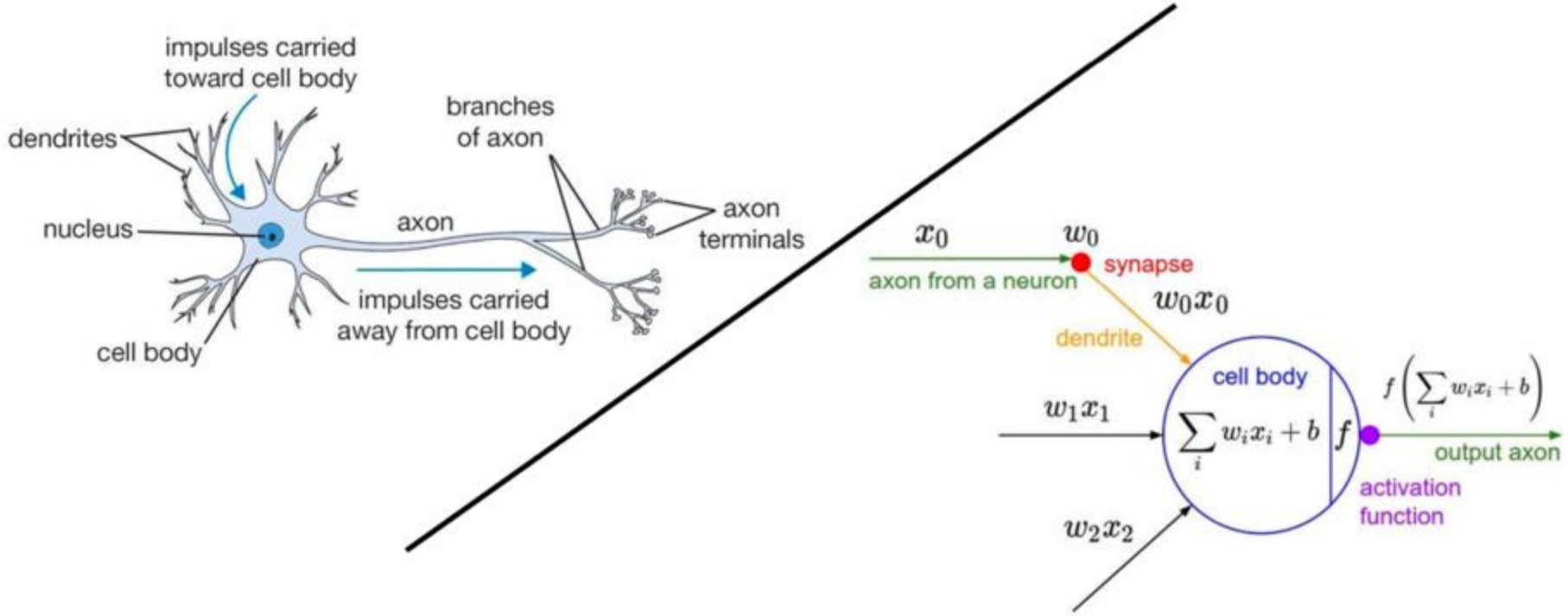
What is Artificial Neural Network?

- **Structure of neural network: Feedforward**



What is Artificial Neural Network?

- **Biological Neurons**



- Artificial neural networks are **inspired** by the brain,
- but not even close in terms of complexity!
- The comparison is great for the media and news articles though...

What is Artificial Neural Network?

- **Biological Neurons**

The New York Times

Microsoft Says New A.I. Shows Signs of Human Reasoning

A provocative paper from researchers at Microsoft claims A.I. technology shows the ability to understand the way people do. Critics say those scientists are kidding themselves.

Bubeck, S., Chandrasekaran, V., Eldan, R., Gehrke, J., Horvitz, E., Kamar, E., ... & Zhang, Y. (**2023**). Sparks of artificial general intelligence: Early experiments with gpt-4. arXiv preprint arXiv:2303.12712.

Sparks of Artificial General Intelligence: Early experiments with GPT-4

Sébastien Bubeck Varun Chandrasekaran Ronen Eldan Johannes Gehrke
Eric Horvitz Ece Kamar Peter Lee Yin Tat Lee Yuanzhi Li Scott Lundberg
Harsha Nori Hamid Palangi Marco Tulio Ribeiro Yi Zhang

Microsoft Research

What is Artificial Neural Network?

- **Summary**

  - Given a dataset with ground truth training pairs $[x_i ; y_i]$

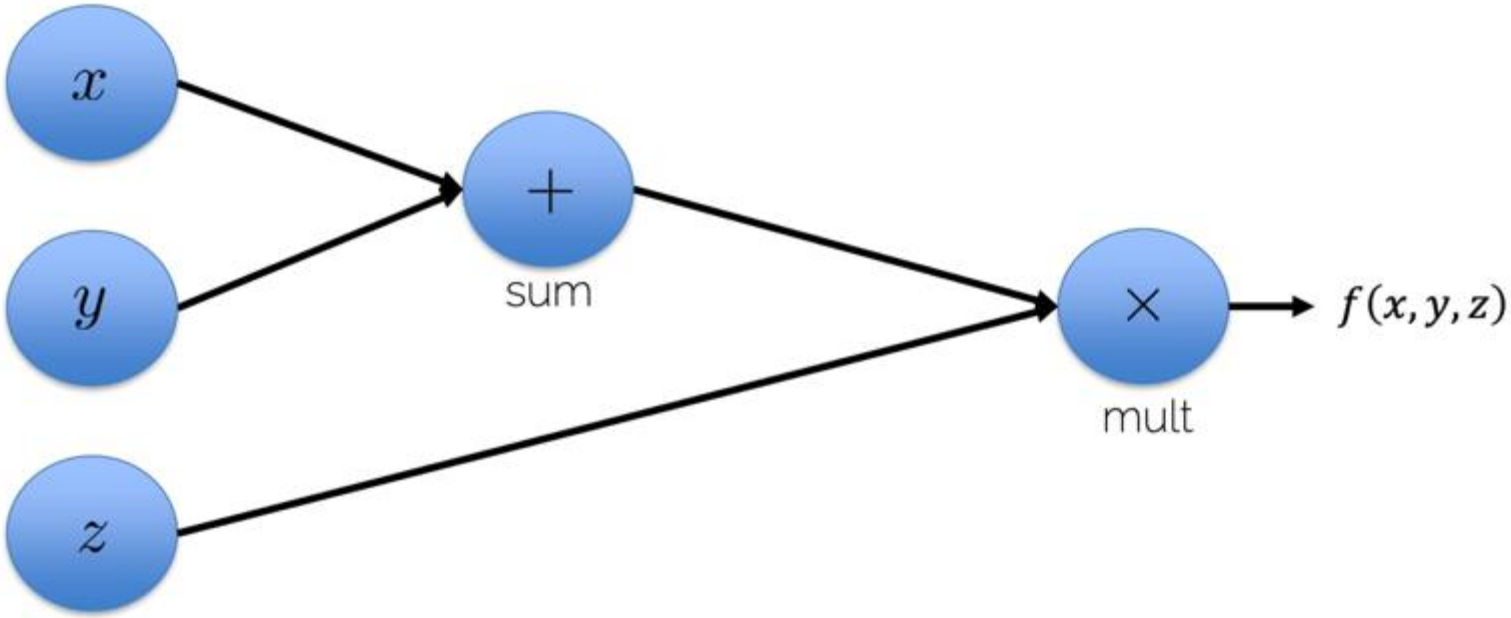
  - Find optimal weights $\boldsymbol{W}$ and biases using stochastic gradient descent, such that the loss function is minimized
    - ***Compute gradients*** with backpropagation (use batch-mode; more later)
    - Iterate many times over training set (SGD; more later)

For Deeper Learning

Computational Graphs

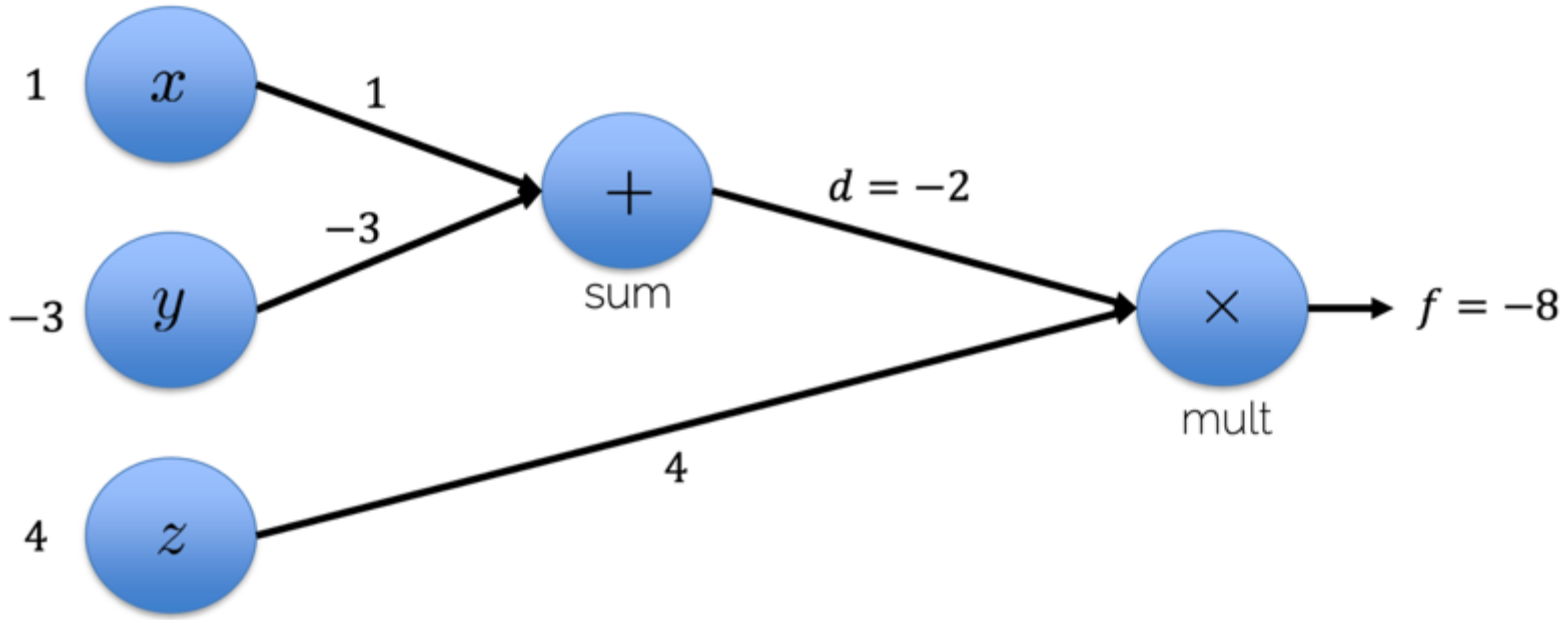
Computational Graphs

- **Directional graph**
- **Matrix operations are represented as compute nodes**
- $f(x,y,z) = (x+y)\cdot z$



Computational Graphs

- **Directional graph**
- **Matrix operations are represented as compute nodes**
- $f(x,y,z) = (x+y)\cdot z$ Initialization x=1, y=−3, z=4



Computational Graphs

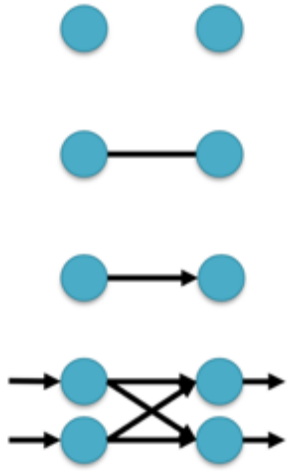
- **Why discuss computational graphs?**

  - Neural networks have complicated architectures
    $\boldsymbol{f} = \boldsymbol{W_5}\,\sigma(\boldsymbol{W_4}\tanh(\boldsymbol{W_3}, \max(\mathbf{0}, \boldsymbol{W_2}\max(\mathbf{0}, \boldsymbol{W_1}\boldsymbol{x}))))$

  - Lot of matrix operations!

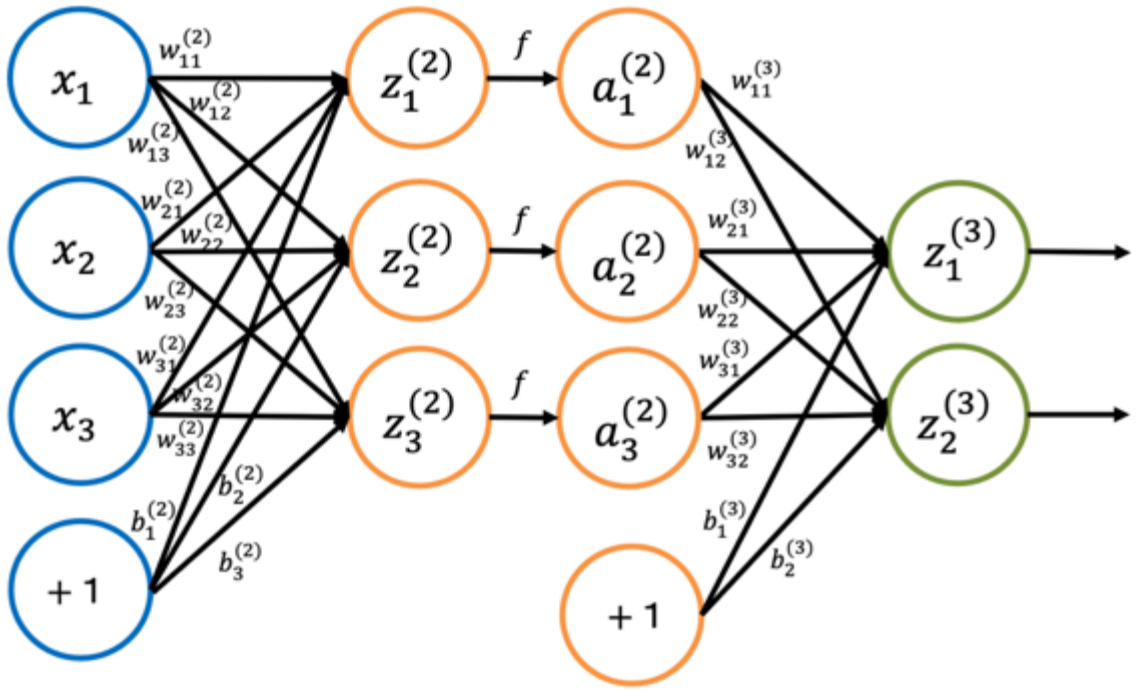
  - *Represent Neural Network (NN) as computational graphs!*

    - NN has compute nodes (operations)
    - NN has edges that connect nodes (data flow)
    - NN is directional
    - NN can be organized into 'layers'



Computational Graphs

- **Why discuss computational graphs?**



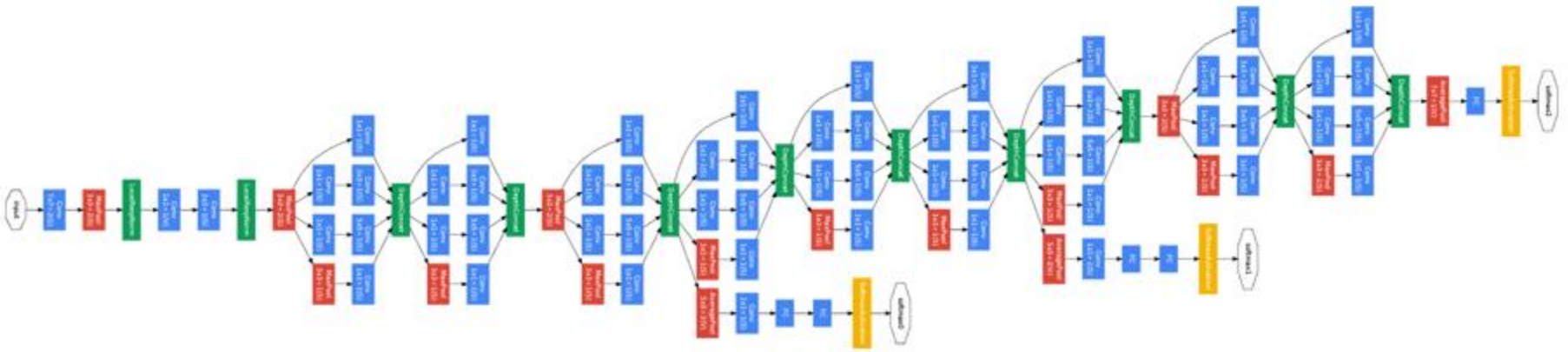
$$z_k^{(2)} = \sum_i x_i w_{ik}^{(2)} + b_k^{(2)}$$

$$a_k^{(2)} = f(z_k^{(2)})$$

$$z_k^{(3)} = \sum_i a_i^{(2)} w_{ik}^{(3)} + b_k^{(3)}$$

Computational Graphs

- **Why discuss computational graphs?**



Szegedy, C., Liu, W., Jia, Y., Sermanet, P., Reed, S., Anguelov, D., ... & Rabinovich, A. (**2015**). Going deeper with convolutions. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 1-9).`

  - From a set of neurons to a Structured Compute Pipeline

Computational Graphs

- **Loss Functions**
  - Measure the goodness of the predictions
    (or equivalently, the network's performance)
  - Regression loss
    - L1 loss $\boldsymbol{L}(\boldsymbol{y}, \hat{\boldsymbol{y}}; \boldsymbol{\theta}) = \frac{1}{n}\sum_i^n \|y_i - \hat{y}_i\|_1$
    - MSE loss $\boldsymbol{L}(\boldsymbol{y}, \hat{\boldsymbol{y}}; \boldsymbol{\theta}) = \frac{1}{n}\sum_i^n \|y_i - \hat{y}_i\|_2^2$
  - Classification loss (for multi-class classification)
    - Cross Entropy loss $E(y, \hat{y}; \theta) = -\sum_{i=1}^{n}\sum_{k=1}^{K}(y_{ik} \cdot \log \hat{y}_{ik})$

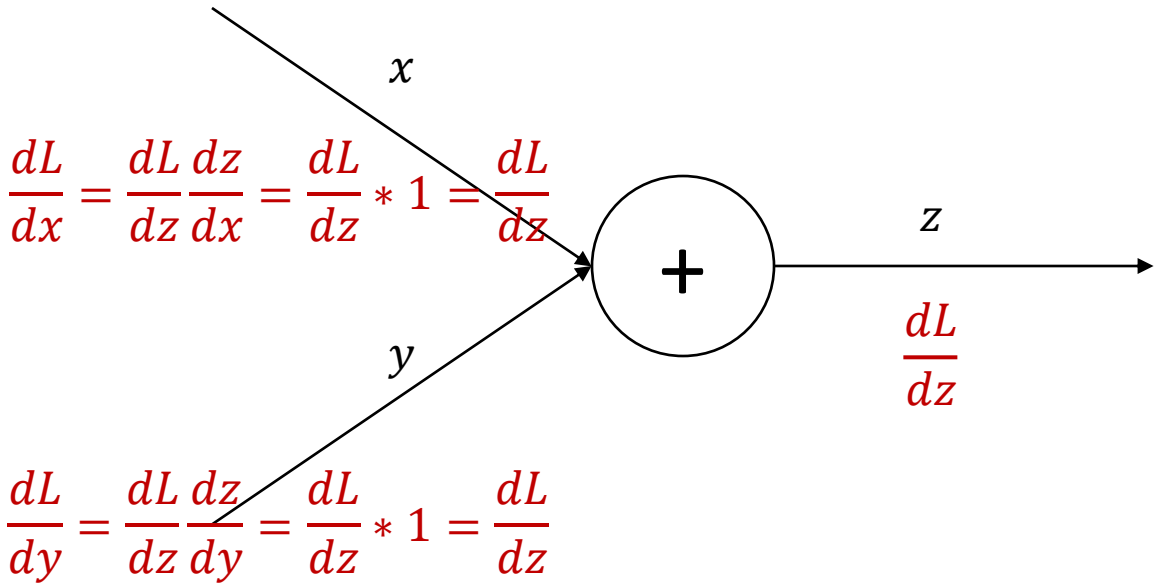
Error Back Propagation

Error Back Propagation

- **Chain Rule**
  - Numerical differentiation is simple and easy to implement,
    but *what if our function is too complex?*
  - **Composite function**: a function made up of multiple functions
    (e.g., $z = (x + y)^2$ is composed of two expressions $z = t^2,\ t = x + y$)
  - The chain rule can express the derivative of a composite function as
    the product of the derivatives of each function constituting the composite function
    → $\frac{dz}{dx} = \frac{dz}{dt}\frac{dt}{dx} = (2t)(1) = 2(x + y)$

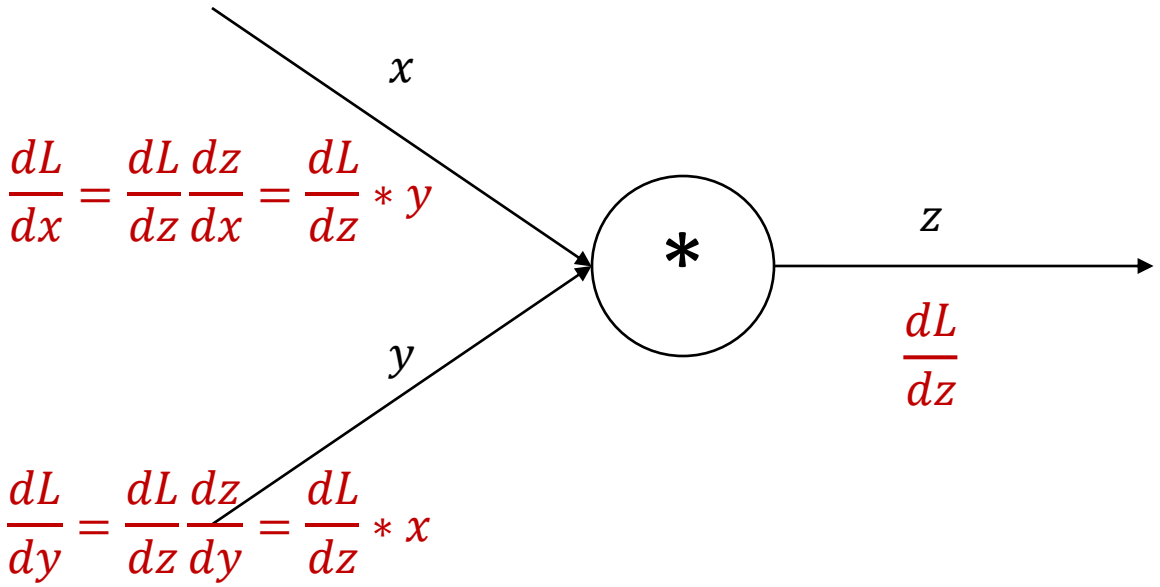
Error Back Propagation

- **Addition Node**
  - Backpropagating the expression $z=x+y$



Error Back Propagation

- **Multiplication Node**
  - Backpropagating the expression $z=xy$



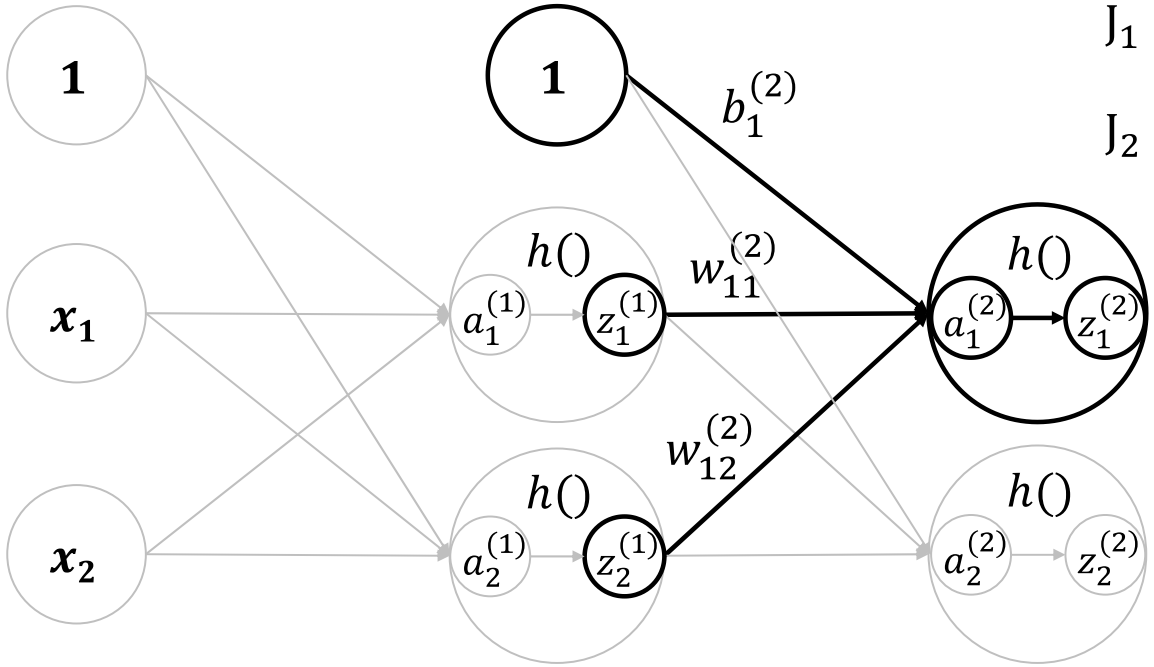
Error Back Propagation

- **Backpropagation Procedure**

$$J_{Total} = J_1 + J_2$$

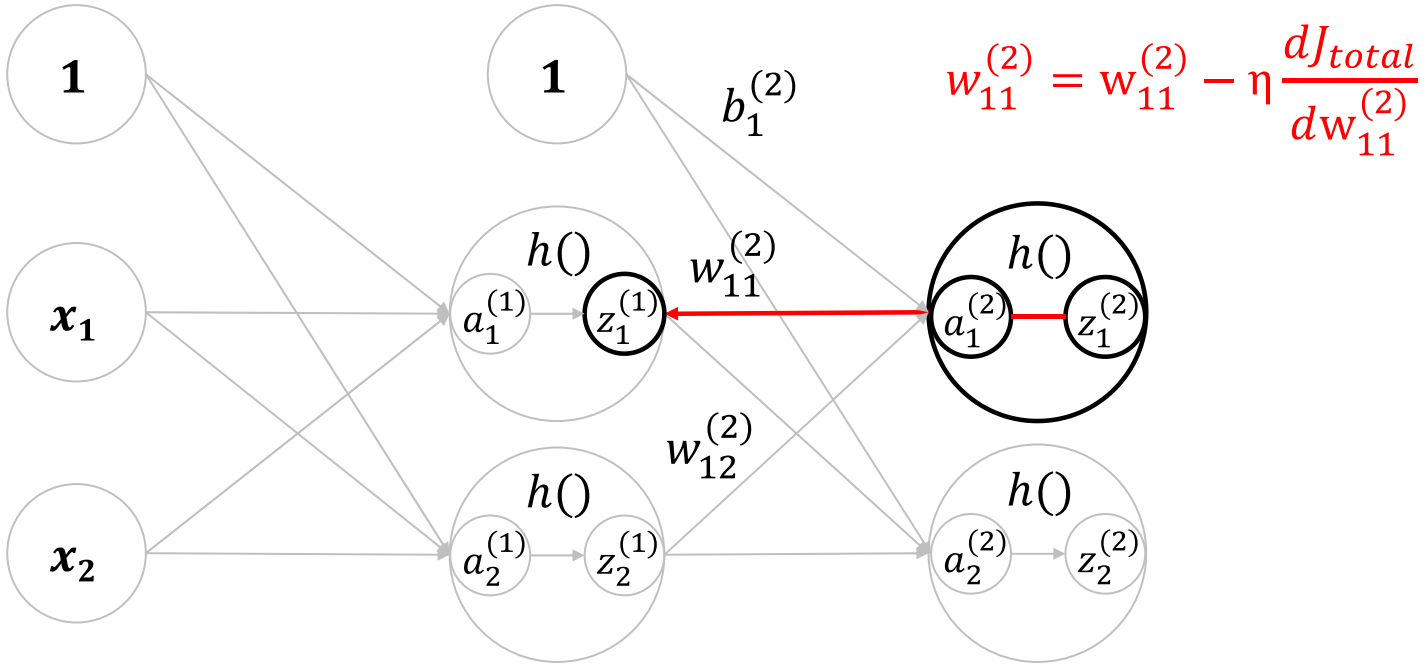
$$J_1 = \frac{1}{2}\left(z_1^{(2)} - y_1\right)^2$$

$$J_2 = \frac{1}{2}\left(z_2^{(2)} - y_2\right)^2$$



Error Back Propagation

- **Backpropagation Procedure**

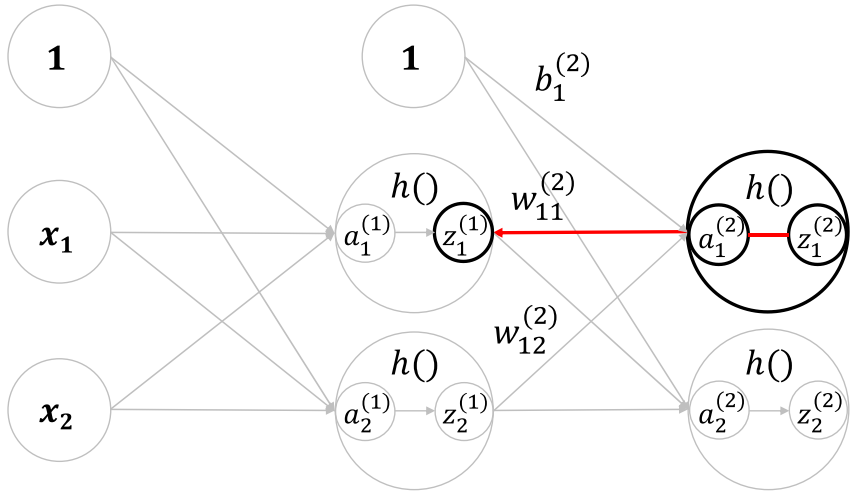


Error Back Propagation

- **Backpropagation Procedure**

$$\frac{dJ_{total}}{dw_{11}^{(2)}} = \frac{dJ_1}{dz_1^{(2)}} \frac{dz_1^{(2)}}{da_1^{(2)}} \frac{da_1^{(2)}}{dw_{11}^{(2)}}$$

$$J_1 = \frac{1}{2}\left(z_1^{(2)} - y_1\right)^2$$
$$z_1^{(2)} = h\left(a_1^{(2)}\right)$$
$$a_1^{(2)} = w_{11}^{(2)} z_1^{(1)} + w_{12}^{(2)} z_2^{(1)} + b_1^{(2)}$$



Error Back Propagation

- **Backpropagation Procedure**

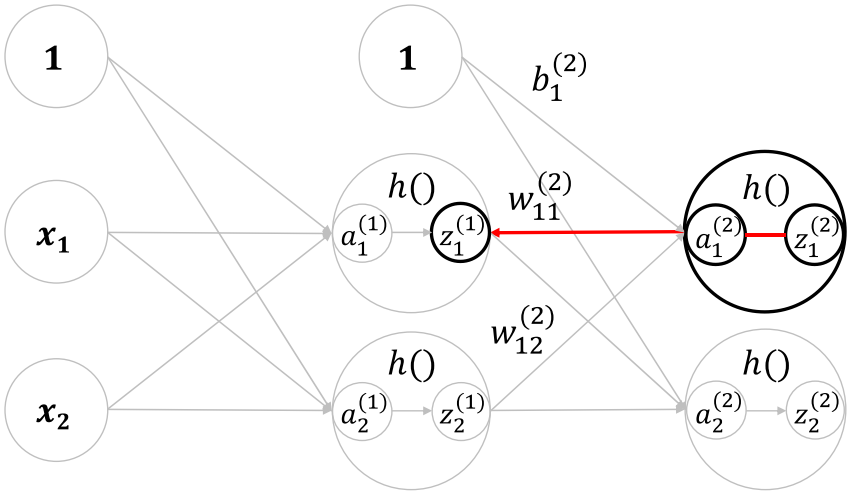
$$\frac{dJ_{total}}{dw_{11}^{(2)}} = \frac{dJ_1}{dz_1^{(2)}} \frac{dz_1^{(2)}}{da_1^{(2)}} \frac{da_1^{(2)}}{dw_{11}^{(2)}}$$

$$\frac{dJ_1}{dz_1^{(2)}} = \left(\frac{1}{2}\right)(2)\left(z_1^{(2)} - y_1\right) = \left(z_1^{(2)} - y_1\right)$$

$$J_1 = \frac{1}{2}\left(z_1^{(2)} - y_1\right)^2$$

$$z_1^{(2)} = h\left(a_1^{(2)}\right)$$

$$a_1^{(2)} = w_{11}^{(2)} z_1^{(1)} + w_{12}^{(2)} z_2^{(1)} + b_1^{(2)}$$



Error Back Propagation

- **Backpropagation Procedure**

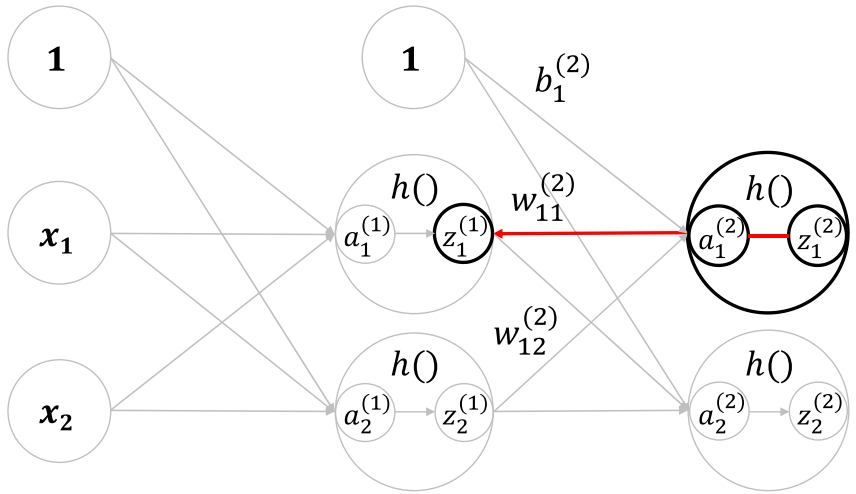
$$\frac{dJ_{total}}{dw_{11}^{(2)}} = \frac{dJ_1}{dz_1^{(2)}} \frac{dz_1^{(2)}}{da_1^{(2)}} \frac{da_1^{(2)}}{dw_{11}^{(2)}}$$

$$\frac{dz_1^{(2)}}{da_1^{(2)}} = h\left(a_1^{(2)}\right)\left(1 - h\left(a_1^{(2)}\right)\right) = z_1^{(2)}\left(1 - z_1^{(2)}\right)$$

$$J_1 = \frac{1}{2}\left(z_1^{(2)} - y_1\right)^2$$

$$z_1^{(2)} = h\left(a_1^{(2)}\right)$$

$$a_1^{(2)} = w_{11}^{(2)} z_1^{(1)} + w_{12}^{(2)} z_2^{(1)} + b_1^{(2)}$$



Error Back Propagation

- **Backpropagation Procedure**

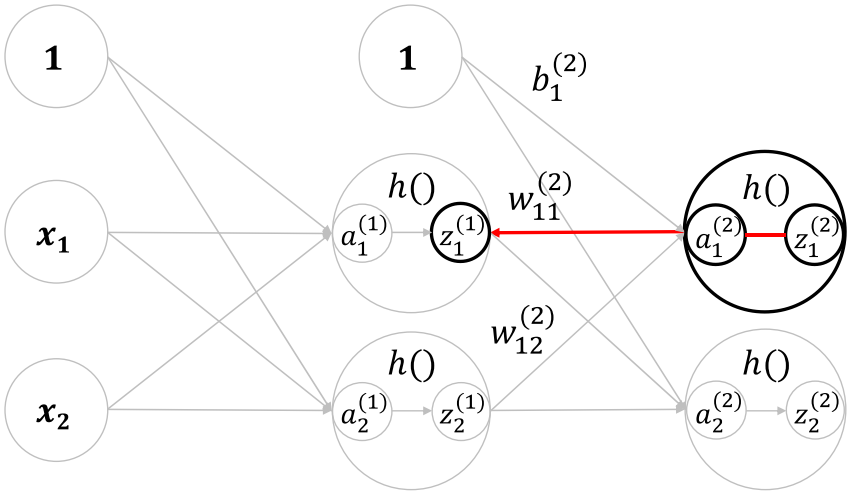
$$\frac{dJ_{total}}{dw_{11}^{(2)}} = \frac{dJ_1}{dz_1^{(2)}} \frac{dz_1^{(2)}}{da_1^{(2)}} \frac{da_1^{(2)}}{dw_{11}^{(2)}}$$

$$\frac{da_1^{(2)}}{dw_{11}^{(2)}} = z_1^{(1)}$$

$$J_1 = \frac{1}{2}\left(z_1^{(2)} - y_1\right)^2$$

$$z_1^{(2)} = h\left(a_1^{(2)}\right)$$

$$a_1^{(2)} = w_{11}^{(2)} z_1^{(1)} + w_{12}^{(2)} z_2^{(1)} + b_1^{(2)}$$



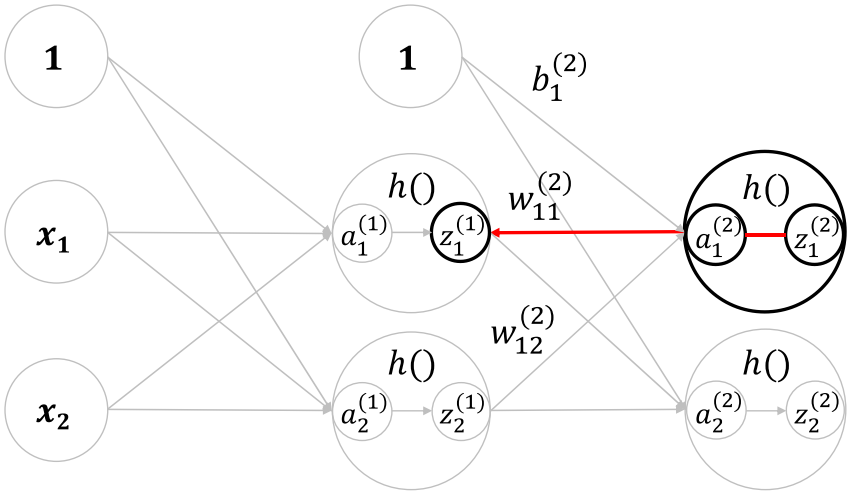
Error Back Propagation

- **Backpropagation Procedure**

$$\frac{dJ_{total}}{dw_{11}^{(2)}} = \frac{dJ_1}{dz_1^{(2)}} \frac{dz_1^{(2)}}{da_1^{(2)}} \frac{da_1^{(2)}}{dw_{11}^{(2)}}$$

$$= \left(z_1^{(2)} - y_1\right) z_1^{(2)} \left(1 - z_1^{(2)}\right) z_1^{(1)}$$

$$= \delta_1^{(2)} z_1^{(1)}$$



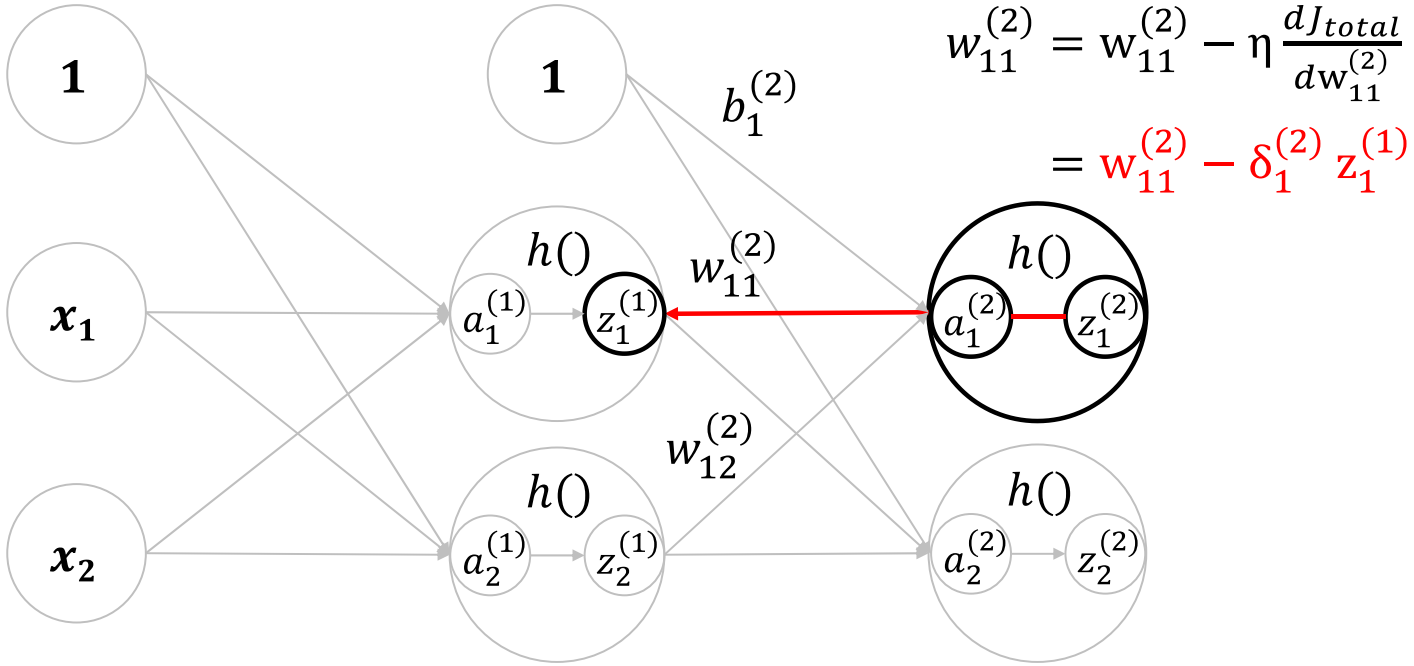
$$\frac{dJ_1}{dz_1^{(2)}} = \left(z_1^{(2)} - y_1\right)$$

$$\frac{dz_1^{(2)}}{da_1^{(2)}} = z_1^{(2)}\left(1 - z_1^{(2)}\right)$$

$$\frac{da_1^{(2)}}{dw_{11}^{(2)}} = z_1^{(1)}$$

Error Back Propagation

- **Backpropagation Procedure**

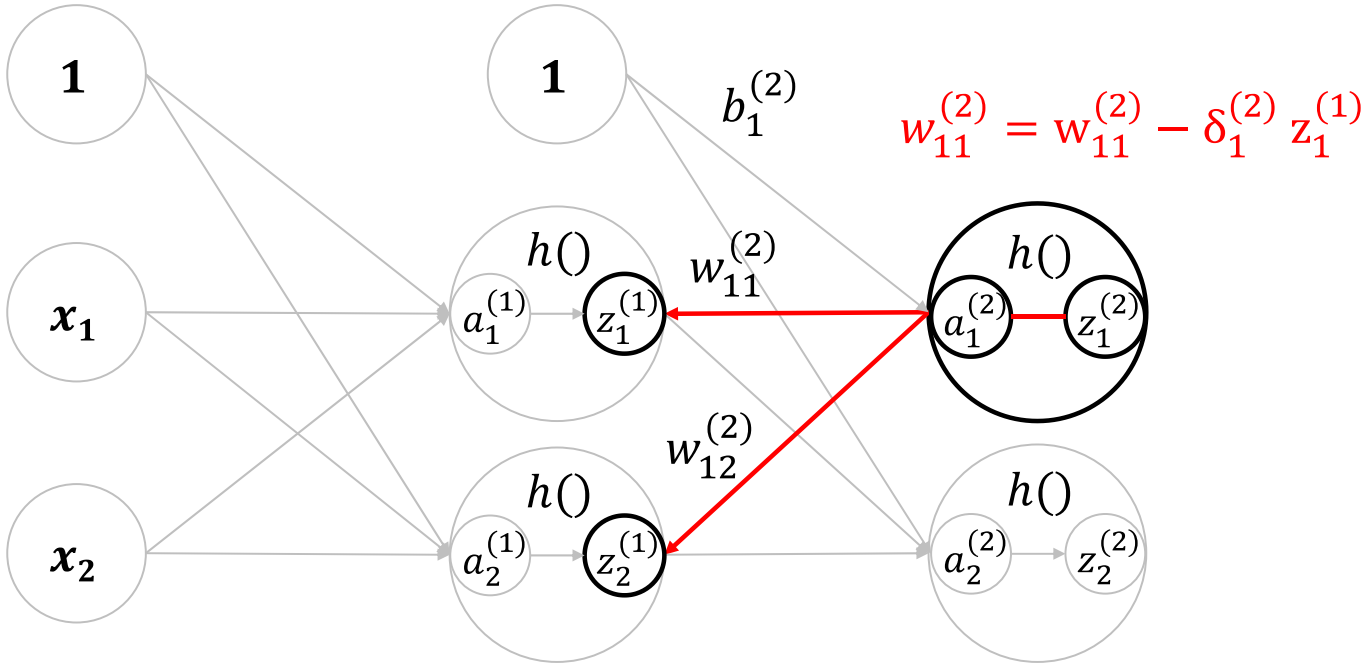


$$w_{11}^{(2)} = w_{11}^{(2)} - \eta \frac{dJ_{total}}{dw_{11}^{(2)}}$$

$$= w_{11}^{(2)} - \delta_1^{(2)} z_1^{(1)}$$

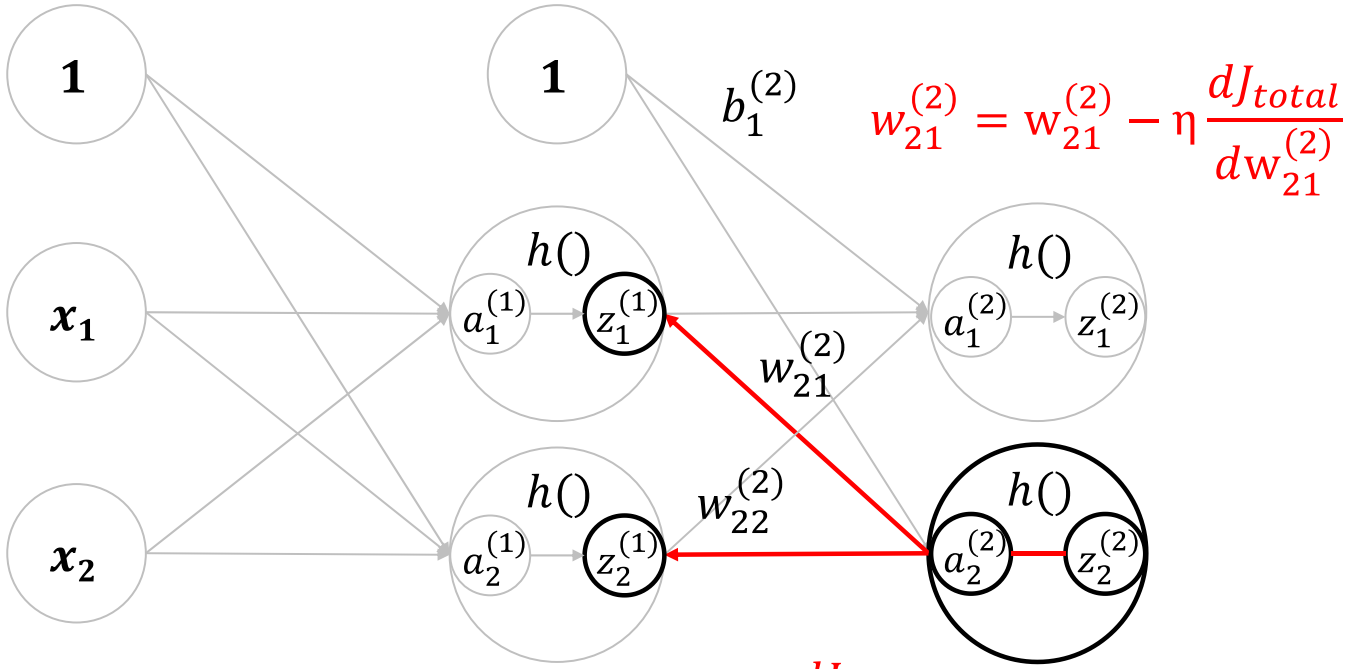
Error Back Propagation

- **Backpropagation Procedure**



Error Back Propagation

- **Backpropagation Procedure**

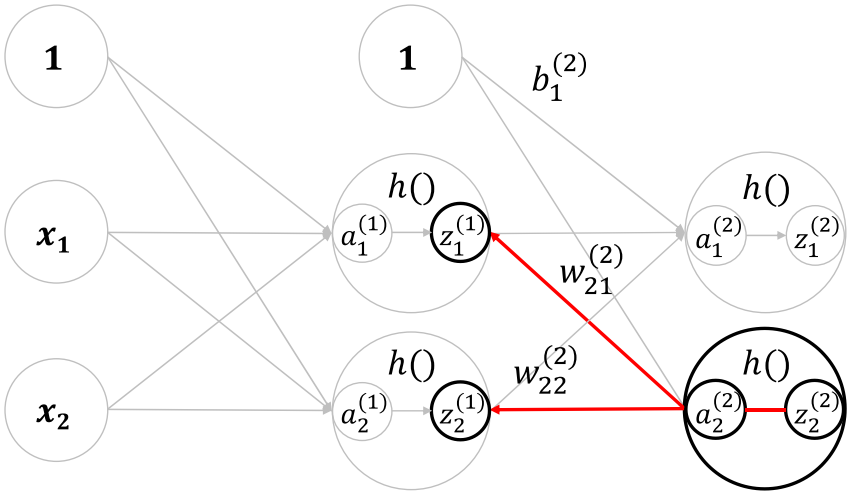


Error Back Propagation

- **Backpropagation Procedure**

$$\frac{dJ_{total}}{dw_{21}^{(2)}} = \frac{dJ_2}{dz_2^{(2)}} \frac{dz_2^{(2)}}{da_2^{(2)}} \frac{da_2^{(2)}}{dw_{21}^{(2)}}$$

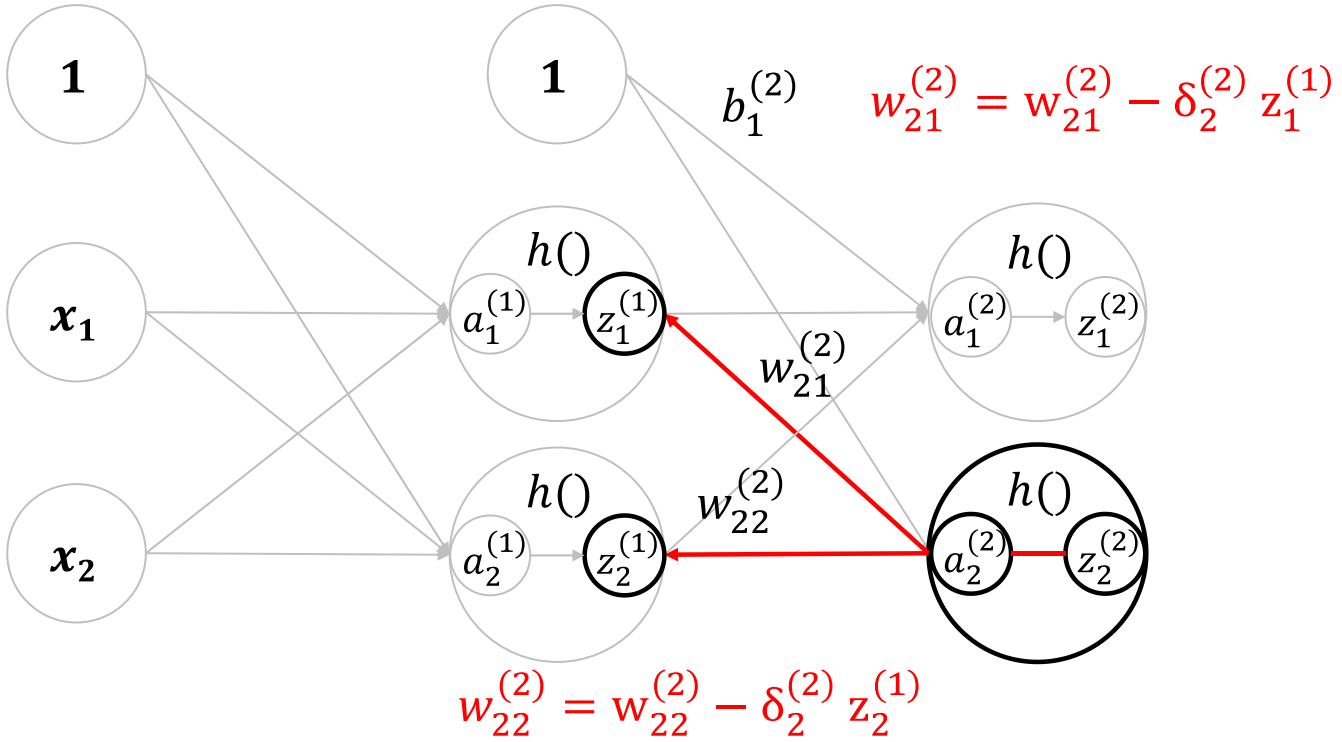
$$\frac{dJ_{total}}{dw_{22}^{(2)}} = \frac{dJ_2}{dz_2^{(2)}} \frac{dz_2^{(2)}}{da_2^{(2)}} \frac{da_2^{(2)}}{dw_{22}^{(2)}}$$



$$J_2 = \frac{1}{2}\left(z_2^{(2)} - y_2\right)^2$$
$$z_2^{(2)} = h\left(a_2^{(2)}\right)$$
$$a_2^{(2)} = w_{21}^{(2)} z_1^{(1)} + w_{22}^{(2)} z_2^{(1)} + b_1^{(2)}$$

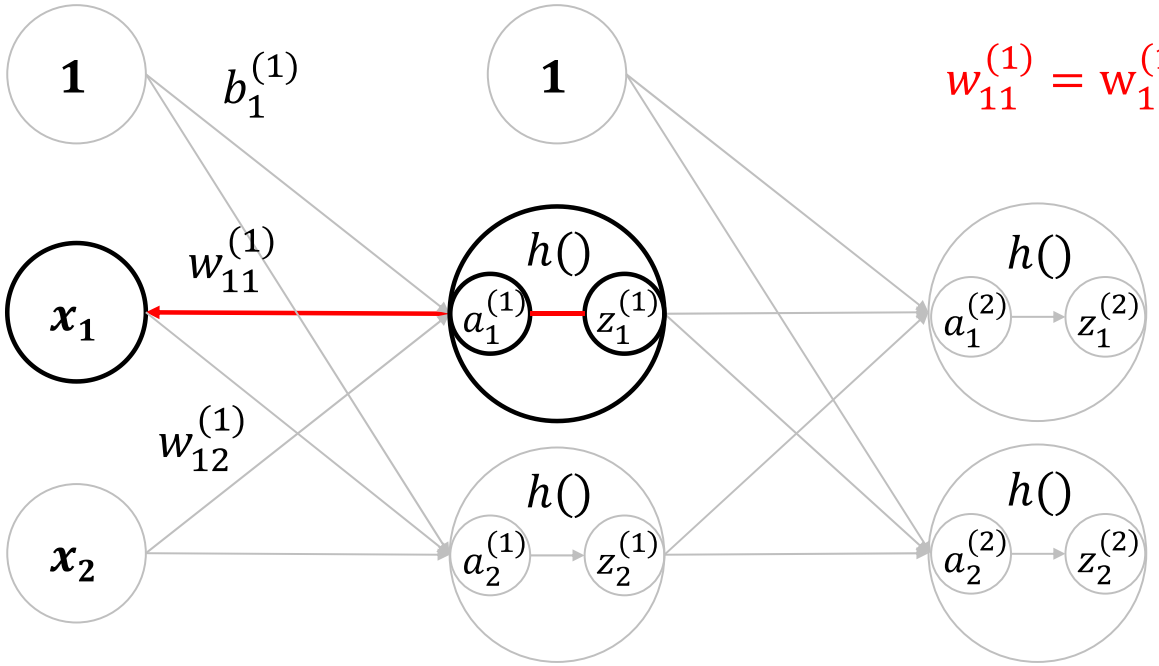
Error Back Propagation

- **Backpropagation Procedure**



Error Back Propagation

- **Backpropagation Procedure**



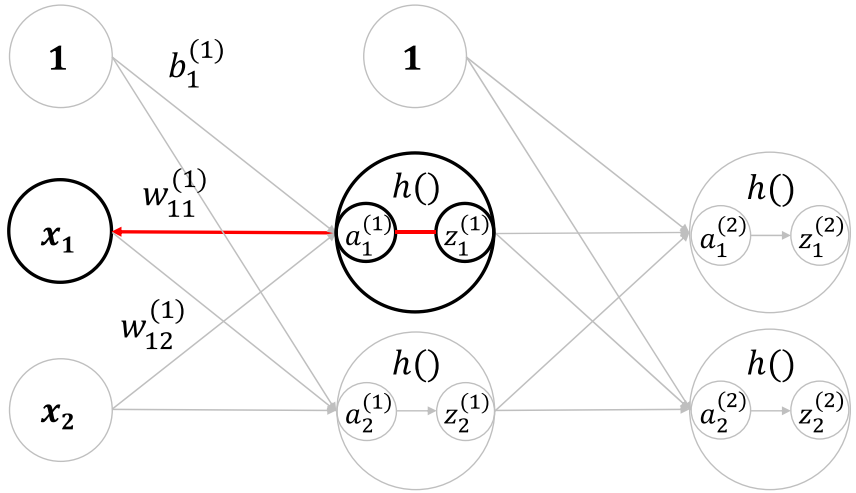
$$w_{11}^{(1)} = \mathrm{w}_{11}^{(1)} - \eta \frac{dJ_{total}}{d\mathrm{w}_{11}^{(1)}}$$

Error Back Propagation

- **Backpropagation Procedure**

$$\frac{dJ_{total}}{dw_{11}^{(1)}} = \frac{dJ_{total}}{dz_1^{(1)}} \frac{dz_1^{(1)}}{da_1^{(1)}} \frac{da_1^{(1)}}{dw_{11}^{(1)}}$$

$$J_1 = \frac{1}{2}\left(z_1^{(2)} - y_1\right)^2$$
$$J_2 = \frac{1}{2}\left(z_2^{(2)} - y_2\right)^2$$
$$z_1^{(1)} = h\left(a_1^{(1)}\right)$$
$$a_1^{(1)} = w_{11}^{(1)} x_1 + w_{12}^{(1)} x_2 + b_1^{(1)}$$



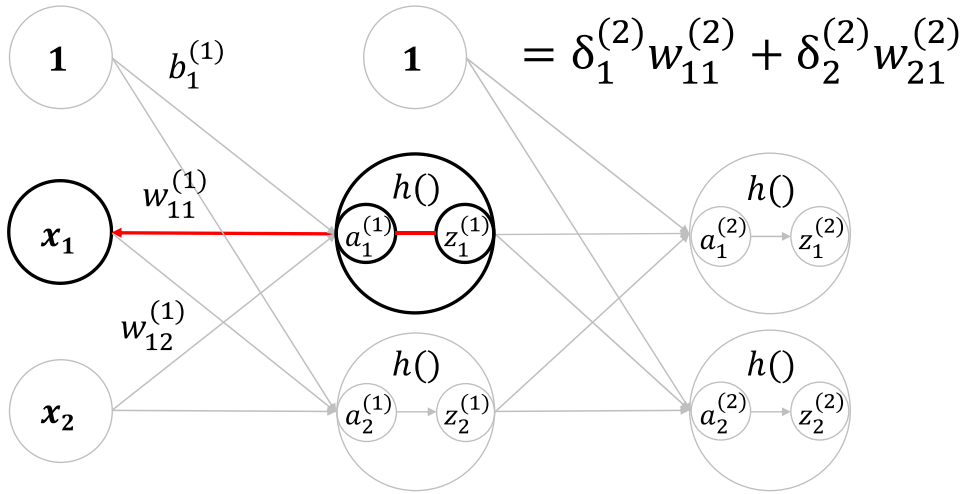
Error Back Propagation

- **Backpropagation Procedure**

$$\frac{dJ_{total}}{dw_{11}^{(1)}} = \frac{dJ_{total}}{dz_1^{(1)}} \frac{dz_1^{(1)}}{da_1^{(1)}} \frac{da_1^{(1)}}{dw_{11}^{(1)}}$$

$$\frac{dJ_{total}}{dz_1^{(1)}} = \frac{dJ_1}{dz_1^{(1)}} + \frac{dJ_2}{dz_1^{(1)}} = \frac{dJ_1}{da_1^{(2)}} \frac{da_1^{(2)}}{dz_1^{(1)}} + \frac{dJ_2}{da_2^{(2)}} \frac{da_2^{(2)}}{dz_1^{(1)}}$$

$$= \delta_1^{(2)} w_{11}^{(2)} + \delta_2^{(2)} w_{21}^{(2)}$$



$$\frac{dJ_1}{da_1^{(2)}} = \delta_1^{(2)}$$

$$\frac{dJ_2}{da_2^{(2)}} = \delta_2^{(2)}$$

$$a_1^{(2)} = w_{11}^{(2)} z_1^{(1)} + w_{12}^{(2)} z_2^{(1)} + b_1^{(2)}$$

$$a_2^{(2)} = w_{21}^{(2)} z_1^{(1)} + w_{22}^{(2)} z_2^{(1)} + b_1^{(2)}$$

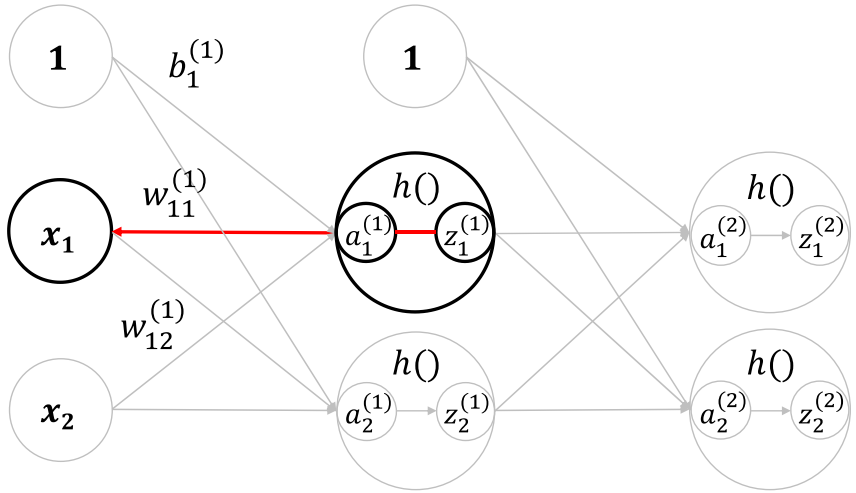
Error Back Propagation

- **Backpropagation Procedure**

$$\frac{dJ_{total}}{dw_{11}^{(1)}} = \frac{dJ_{total}}{dz_1^{(1)}} \frac{dz_1^{(1)}}{da_1^{(1)}} \frac{da_1^{(1)}}{dw_{11}^{(1)}}$$

$$\frac{dz_1^{(1)}}{da_1^{(1)}} = h\left(a_1^{(1)}\right)\left(1 - h\left(a_1^{(1)}\right)\right) = z_1^{(1)}\left(1 - z_1^{(1)}\right)$$

$$z_1^{(1)} = h\left(a_1^{(1)}\right)$$



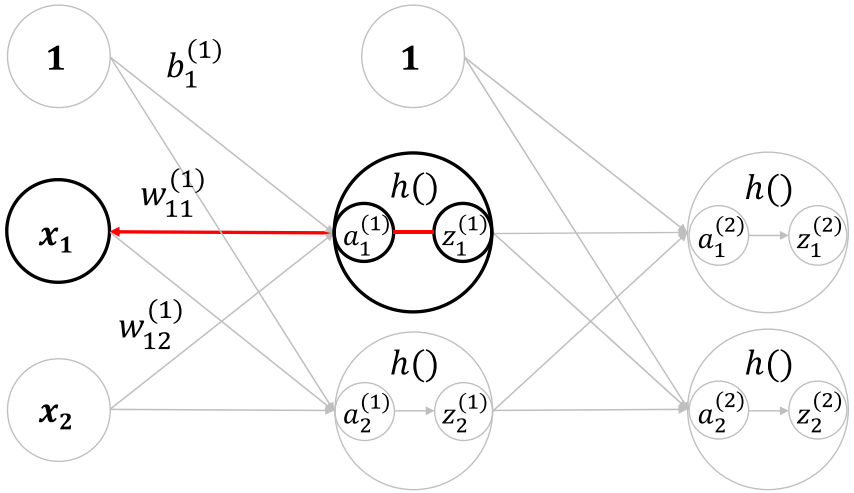
Error Back Propagation

- **Backpropagation Procedure**

$$\frac{dJ_{total}}{dw_{11}^{(1)}} = \frac{dJ_{total}}{dz_1^{(1)}} \frac{dz_1^{(1)}}{da_1^{(1)}} \frac{da_1^{(1)}}{dw_{11}^{(1)}}$$

$$\frac{da_1^{(1)}}{dw_{11}^{(1)}} = x_1$$

$$a_1^{(1)} = w_{11}^{(1)} x_1 + w_{12}^{(1)} x_2 + b_1^{(1)}$$



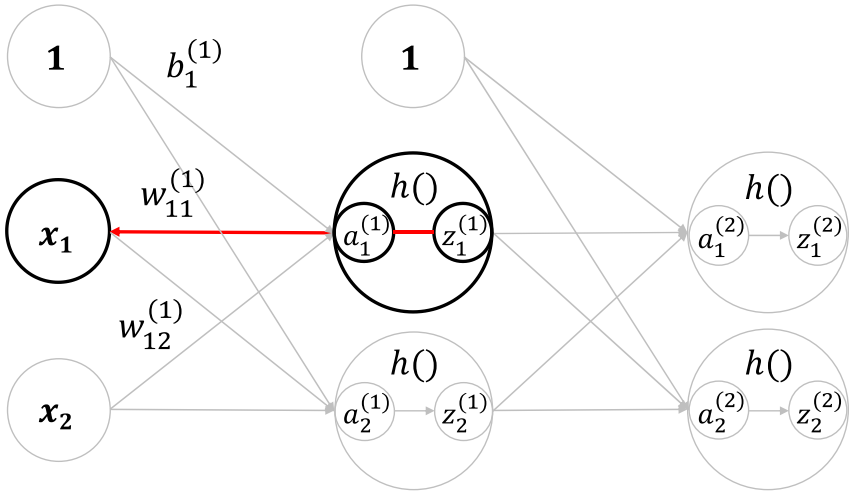
Error Back Propagation

- **Backpropagation Procedure**

$$\frac{dJ_{total}}{dw_{11}^{(1)}} = \frac{dJ_{total}}{dz_1^{(1)}}\frac{dz_1^{(1)}}{da_1^{(1)}}\frac{da_1^{(1)}}{dw_{11}^{(1)}}$$

$$= (\delta_1^{(2)} w_{11}^{(2)} + \delta_2^{(2)} w_{21}^{(2)})(z_1^{(1)}\left(1 - z_1^{(1)}\right))x_1$$

$$= \delta_1^{(1)}\, x_1$$



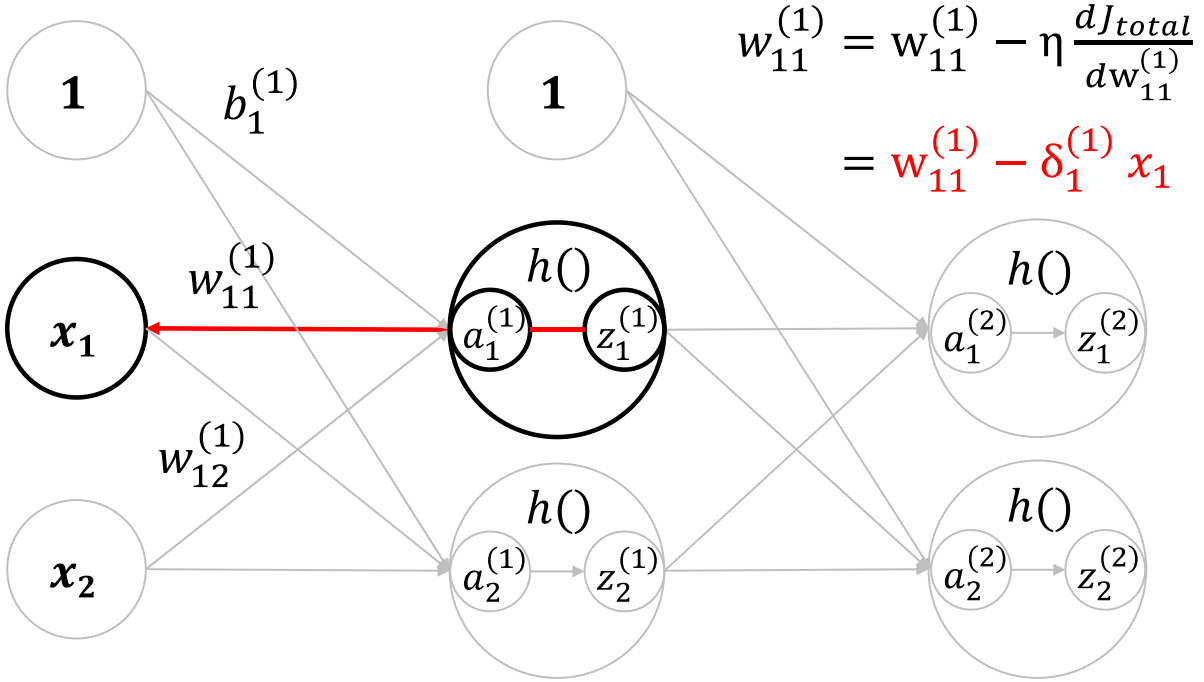
$$\frac{dJ_{total}}{dz_1^{(1)}} = \delta_1^{(2)} w_{11}^{(2)} + \delta_2^{(2)} w_{21}^{(2)}$$

$$\frac{dz_1^{(1)}}{da_1^{(1)}} = z_1^{(1)}\left(1 - z_1^{(1)}\right)$$

$$\frac{da_1^{(1)}}{dw_{11}^{(1)}} = x_1$$

Error Back Propagation

- **Backpropagation Procedure**



$$w_{11}^{(1)} = \mathrm{w}_{11}^{(1)} - \eta \frac{dJ_{total}}{d\mathrm{w}_{11}^{(1)}}$$

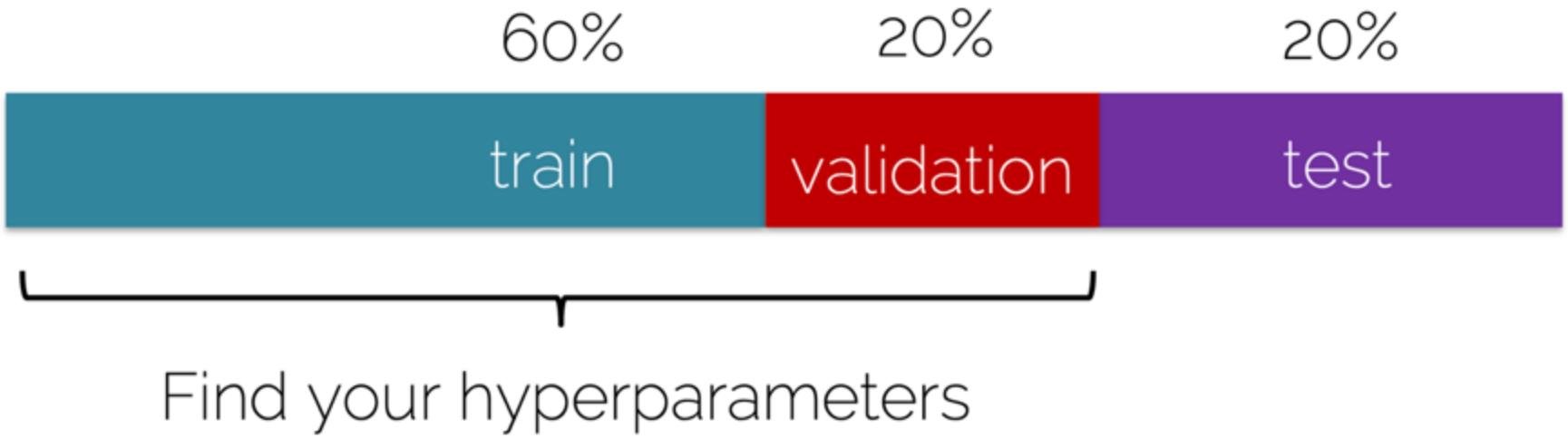
$$= \mathrm{w}_{11}^{(1)} - \delta_1^{(1)} x_1$$

Deep Learning

Regularization

Regularization

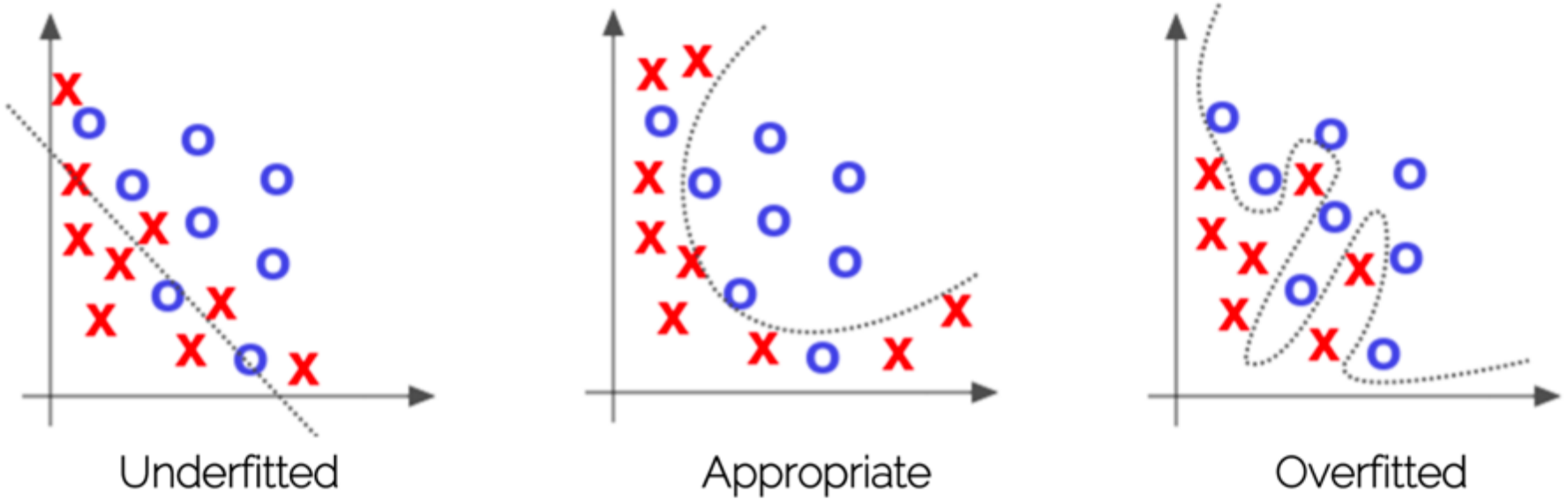
- **Basic Recipe for ML**
  - Split your data



  - Other splits are also possible (e.g., 80% / 10% / 10%)

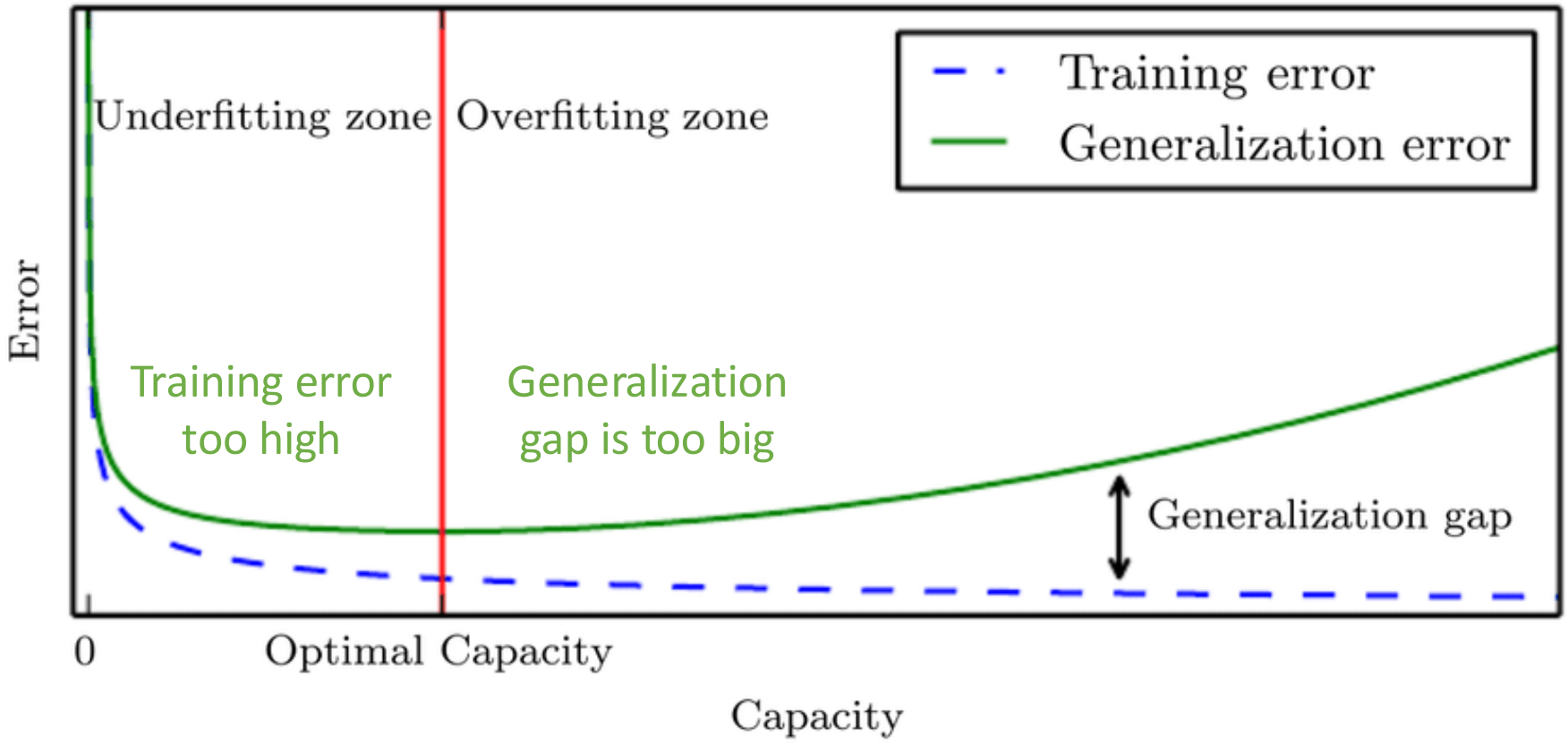
Regularization

- **Over- and Underfitting**



Regularization

- **Training / Validation curve**
  - How can we prevent our model from overfitting?
    → ***Regularization***



Regularization

- **Loss function** $L(\boldsymbol{y}, \hat{\boldsymbol{y}}, \boldsymbol{\theta}) = \sum_{i=1}^{n} (\hat{y}_i - y_i)^2 + \lambda R(\boldsymbol{\theta})$

- **Regularization techniques**
  - L2 regularization
  - L1 regularization

  Add ***regularization term*** to loss function
  - Max norm regularization
  - Dropout
  - Early stopping
  - …

Regularization: Example

- **Input: 3 features $\boldsymbol{x}$ = [1, 2, 1]**

- **Two linear classifiers that give the same result:**

- θ_1 **= [0, 0.75, 0]** → Ignores 2 features

- θ_2 **= [0.25, 0. 5, 0.25]** → Takes information from all features

Regularization: Example

- Loss $L(\boldsymbol{y}, \hat{\boldsymbol{y}}, \boldsymbol{\theta}) = \sum_{i=1}^{n} \left(x_i \theta_{ji} - y_i\right)^2 + \lambda R(\boldsymbol{\theta})$
- L1 regularization $R(\boldsymbol{\theta}) = \sum_{i=1}^{n} |\theta_i|$

$\theta_1 \longrightarrow 0 + 0.75 + 0 = 0.75$ Minimization

$\theta_2 \longrightarrow 0.25 + 0.5 + 0.25 = 1$

$x = [1, 2, 1], \theta_1 = [0, 0.75, 0], \theta_2 = [0.25, 0.5, 0.25]$

Regularization: Example

- Loss $L(\boldsymbol{y}, \hat{\boldsymbol{y}}, \boldsymbol{\theta}) = \sum_{i=1}^{n} \left(x_i \theta_{ji} - y_i\right)^2 + \lambda R(\boldsymbol{\theta})$
- L2 regularization $R(\boldsymbol{\theta}) = \sum_{i=1}^{n} \theta_i^2$

$\theta_1 \longrightarrow 0 + 0.75^2 + 0 = 0.5625$

$\theta_2 \longrightarrow 0.25^2 + 0.5^2 + 0.25^2 = 0.375$ Minimization

$x = [1, 2, 1], \theta_1 = [0, 0.75, 0], \theta_2 = [0.25, 0.5, 0.25]$

Regularization: Example

- **Input: 3 features $\boldsymbol{x}$ = [1, 2, 1]**

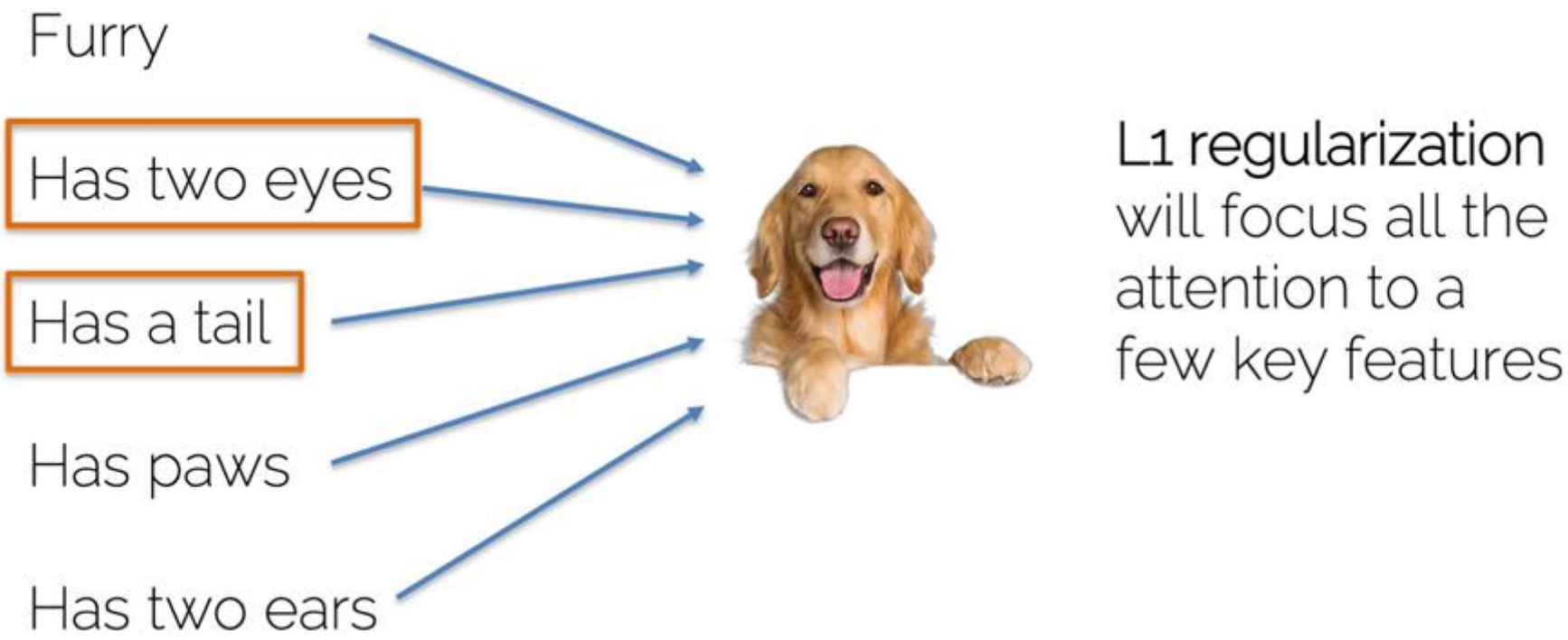
- **Two linear classifiers that give the same result:**

- θ_1 **= [0, 0.75, 0]** → L1 regularization enforces ***sparsity***

- θ_2 **= [0.25, 0. 5, 0.25]** → L2 regularization enforces that the weights have ***similar values***

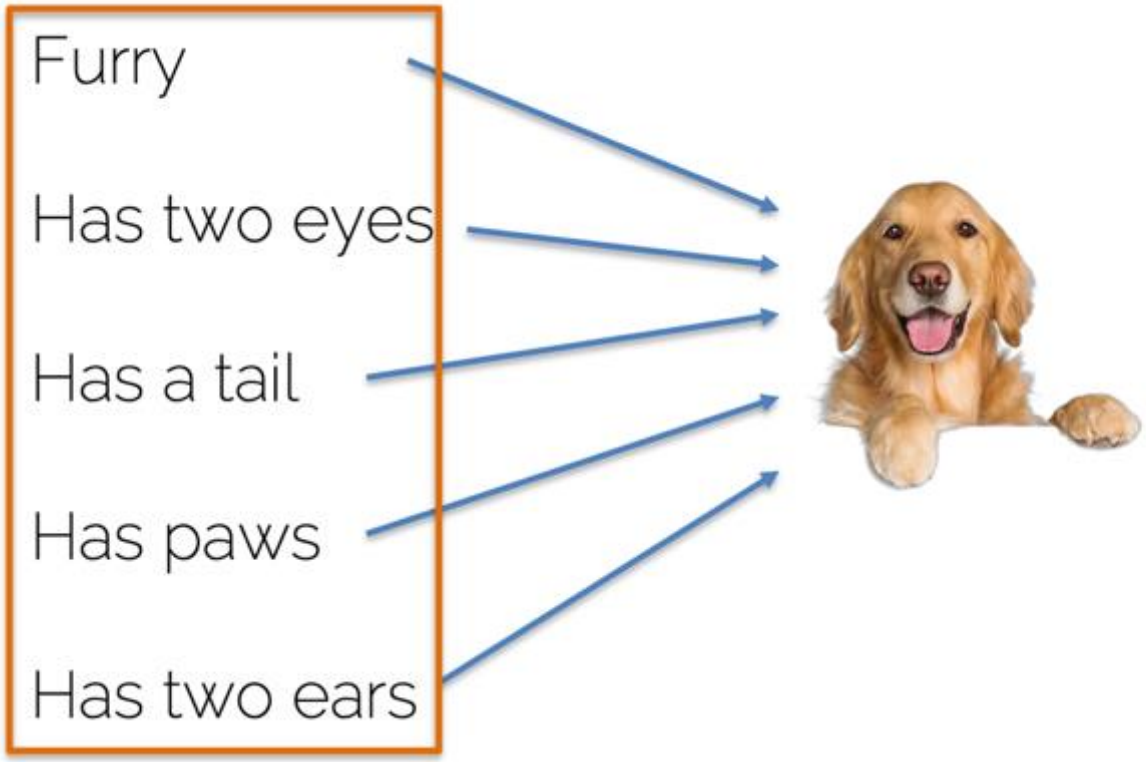
Regularization: Effect

- **Dog classifier takes different inputs**



Regularization: Effect

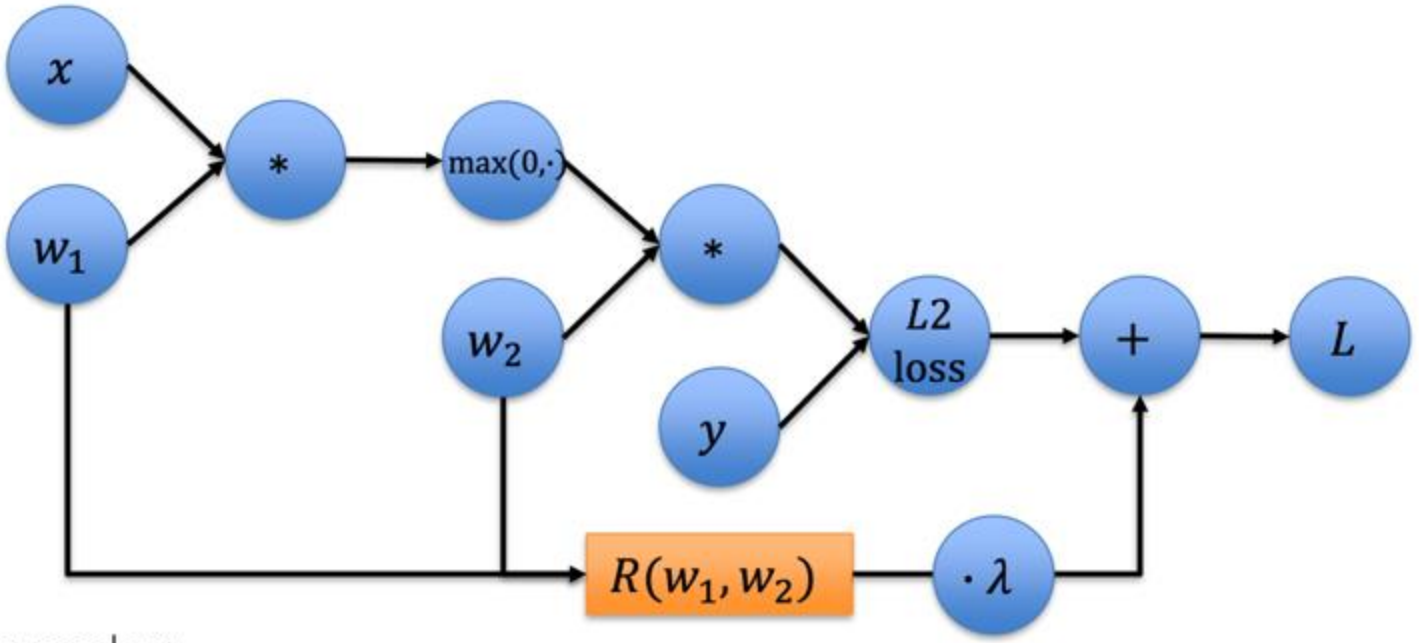
- **Dog classifier takes different inputs**



L2 regularization will take all information into account to make decisions

Regularization

- **Regularization for Neural Networks**

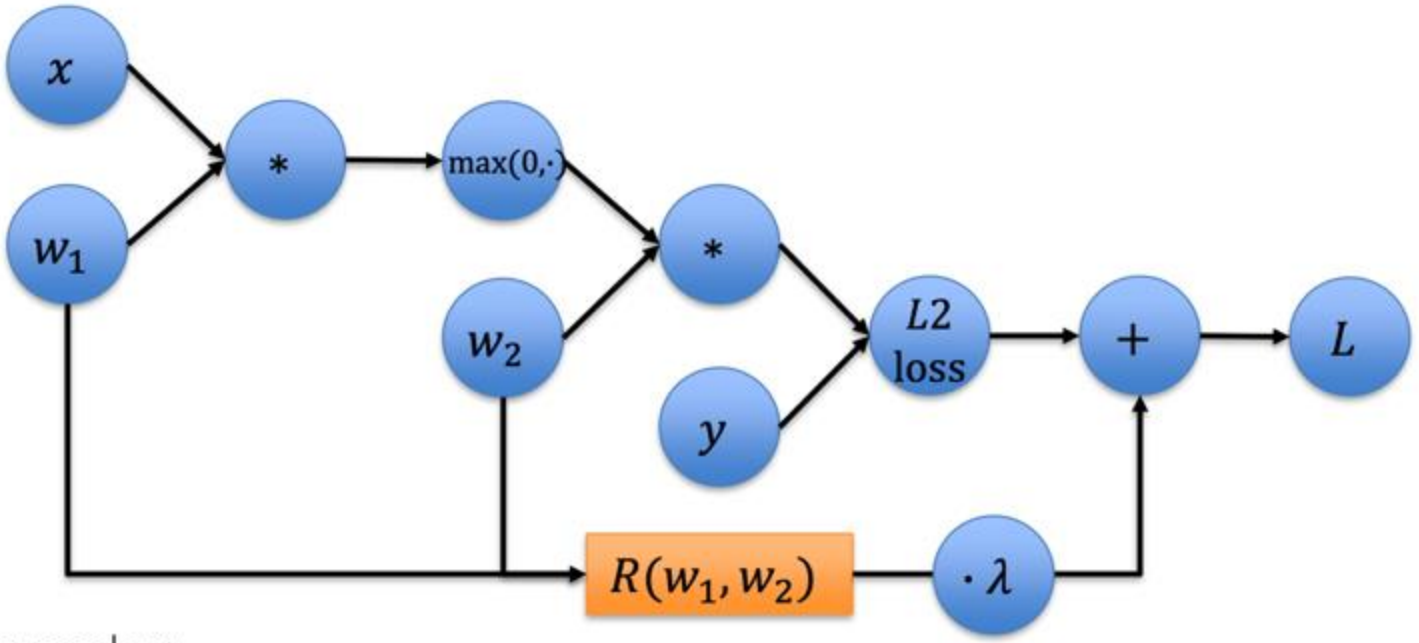


Combining nodes:
Network output + L2-loss + regularization

$$\sum_{i=1}^{n} \|w_2 \max(0, w_1 x_i) - y_i\|_2^2 + \lambda R(w_1, w_2)$$

Regularization

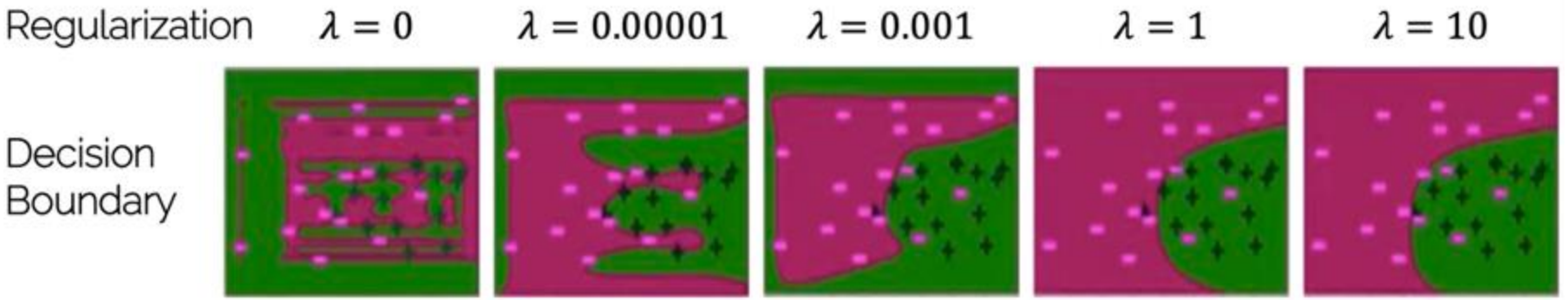
- **Regularization for Neural Networks**



Combining nodes:
Network output + L2-loss + regularization

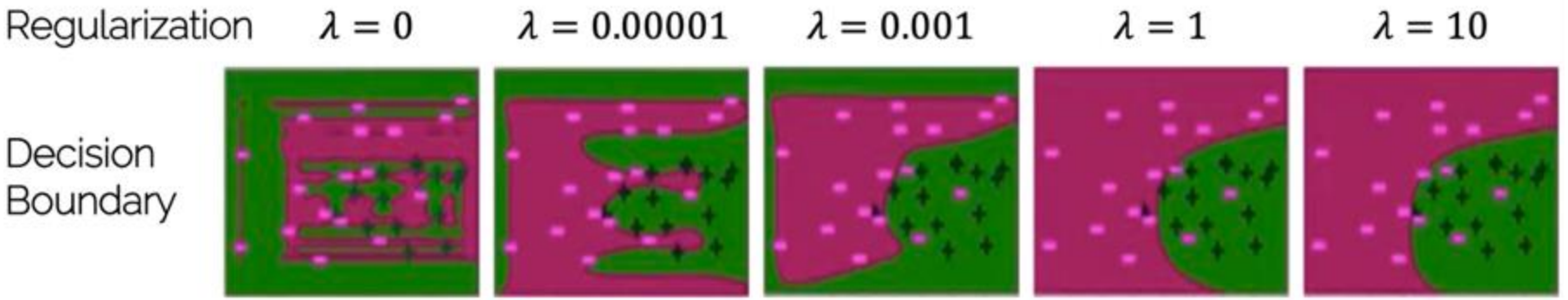
$$\sum_{i=1}^{n} \|w_2 \max(0, w_1 x_i) - y_i\|_2^2 + \lambda(w_1^2 + w_2^2)$$

Regularization



- **What happens to the training error?**
- **What is the goal of regularization?**

Regularization



- **What happens to the training error?**
- **What is the goal of regularization?**

Lower validation error

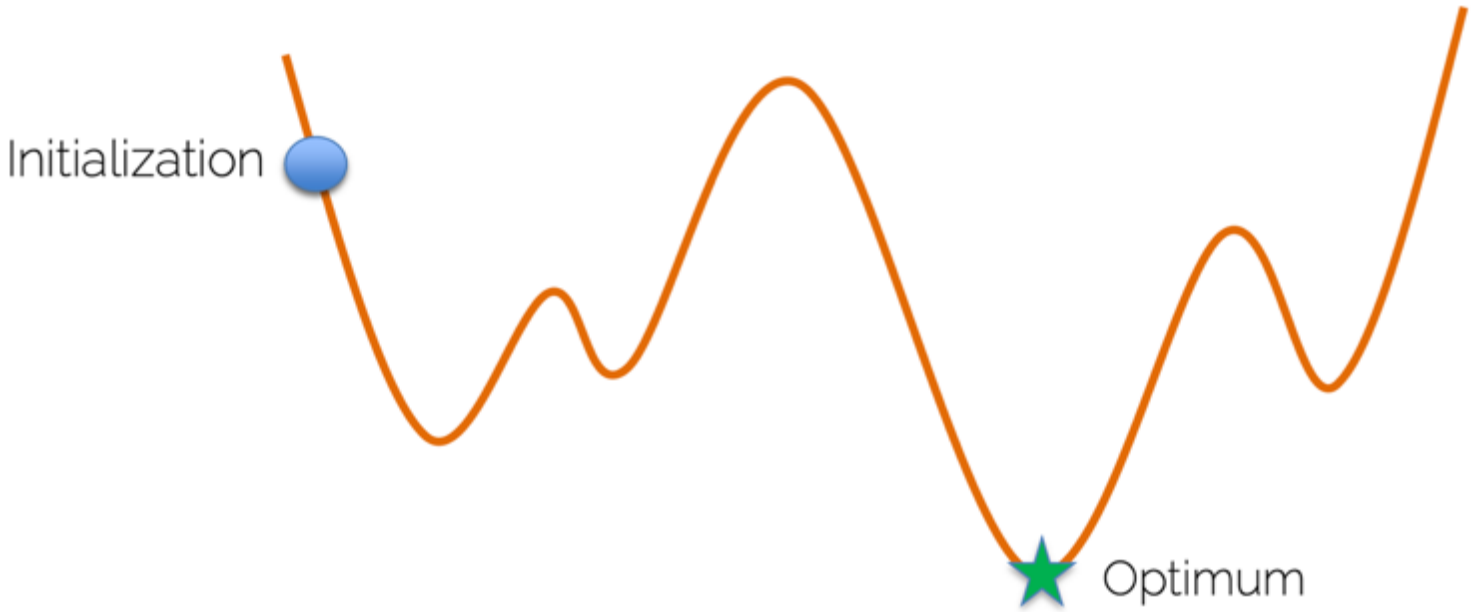
Increasing training error

Optimization

Optimization

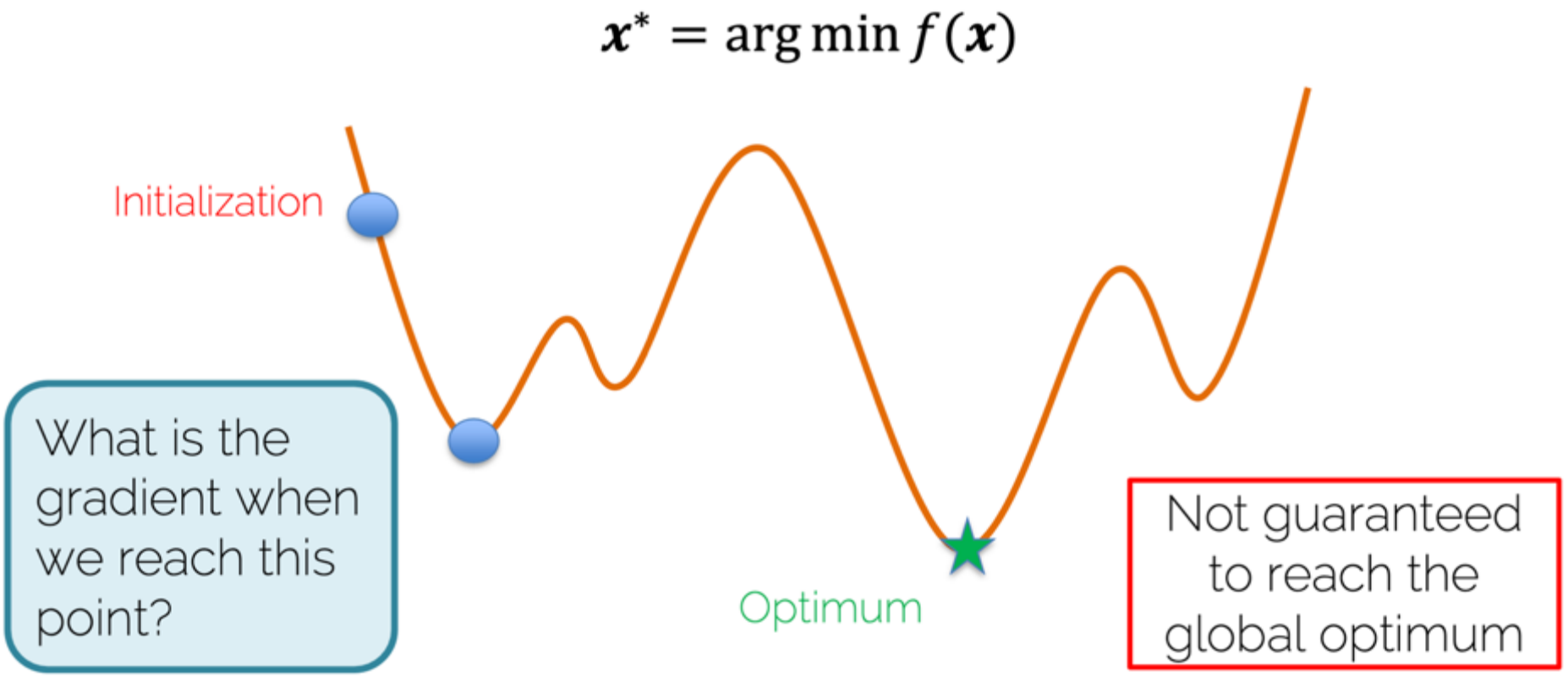
- **Gradient Descent**

$$x^* = \arg\min f(x)$$



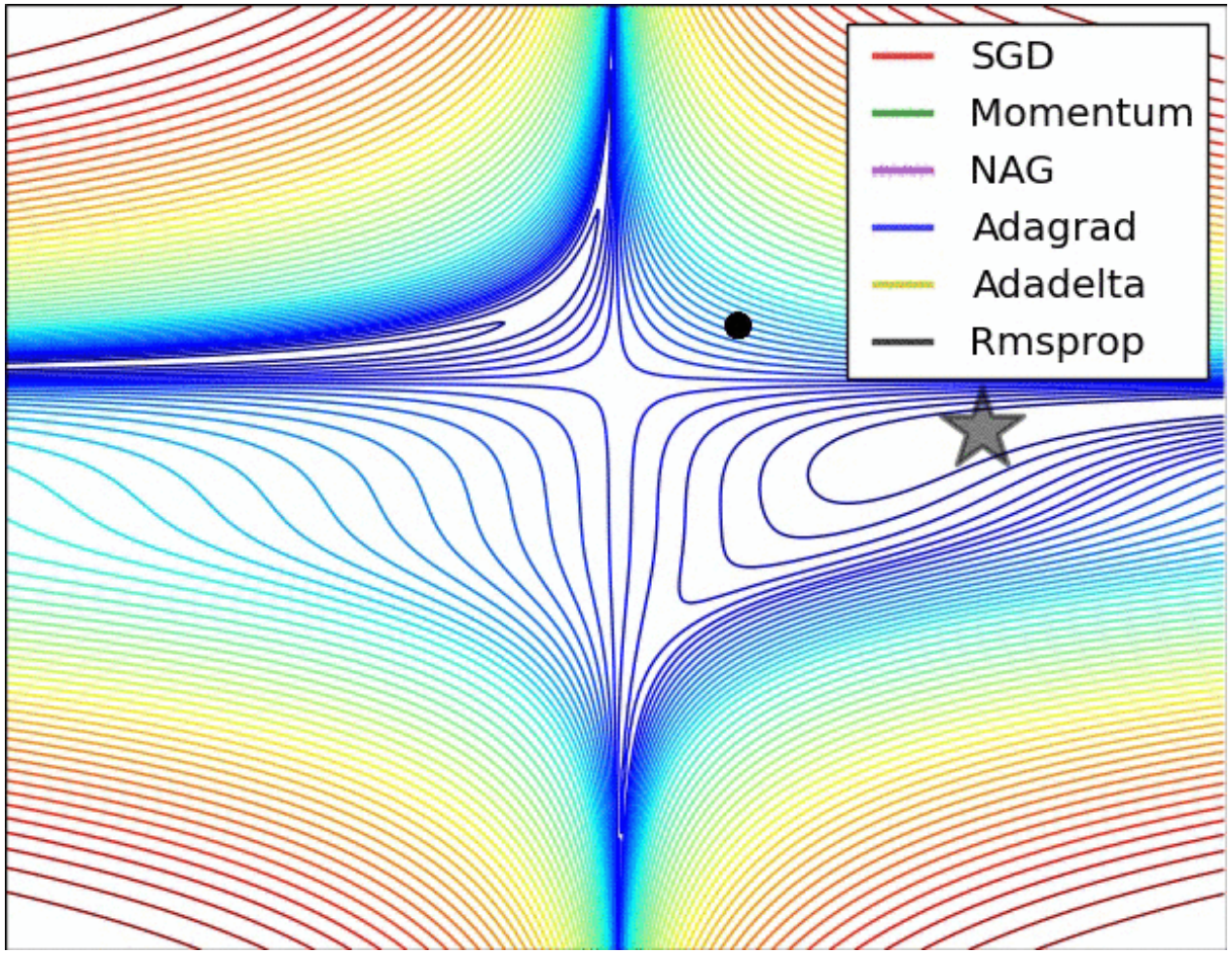
Optimization

- **Gradient Descent**



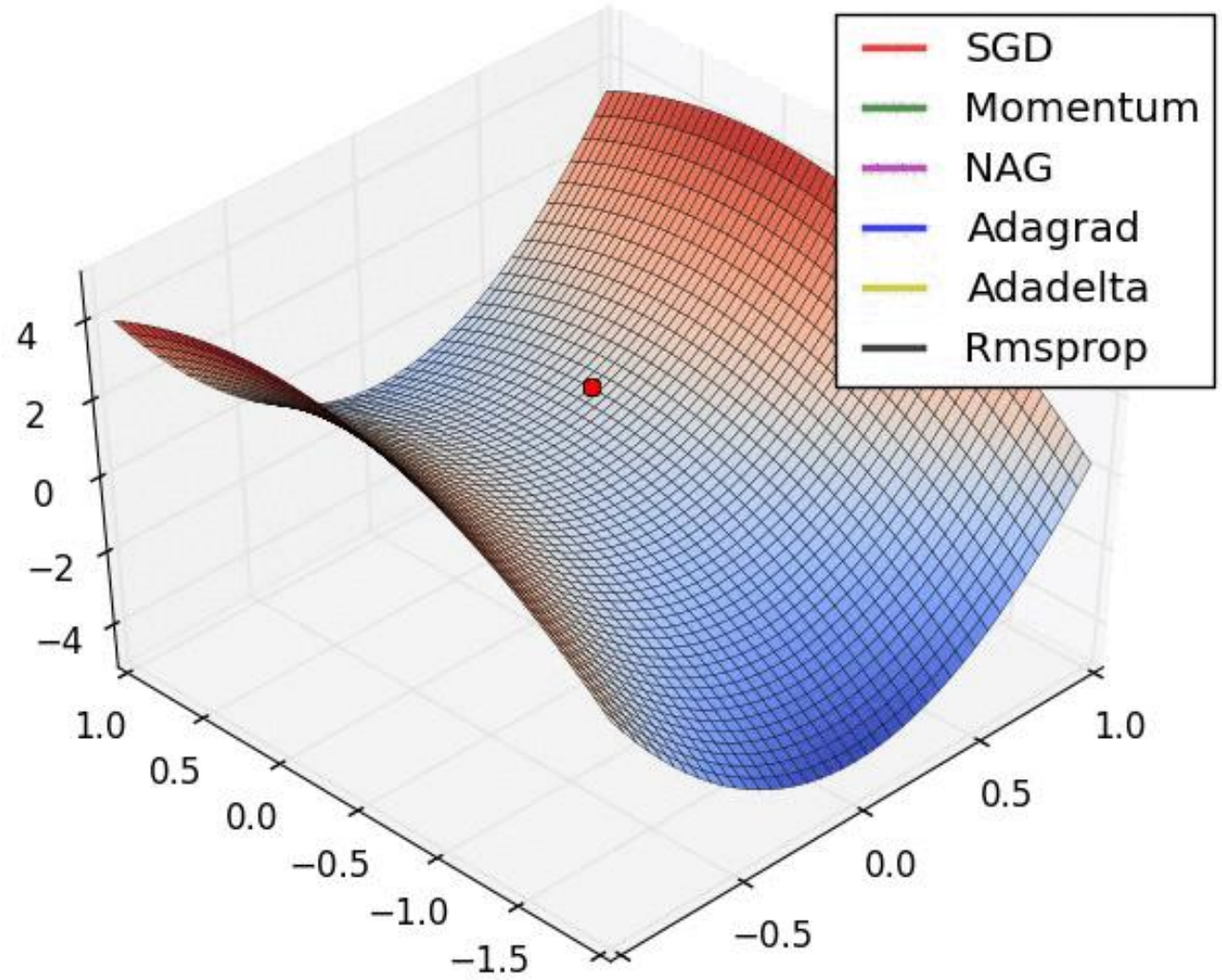
Optimization

- **Adam**
  - A new method proposed in 2015 by combining the strengths of Momentum + AdaGrad



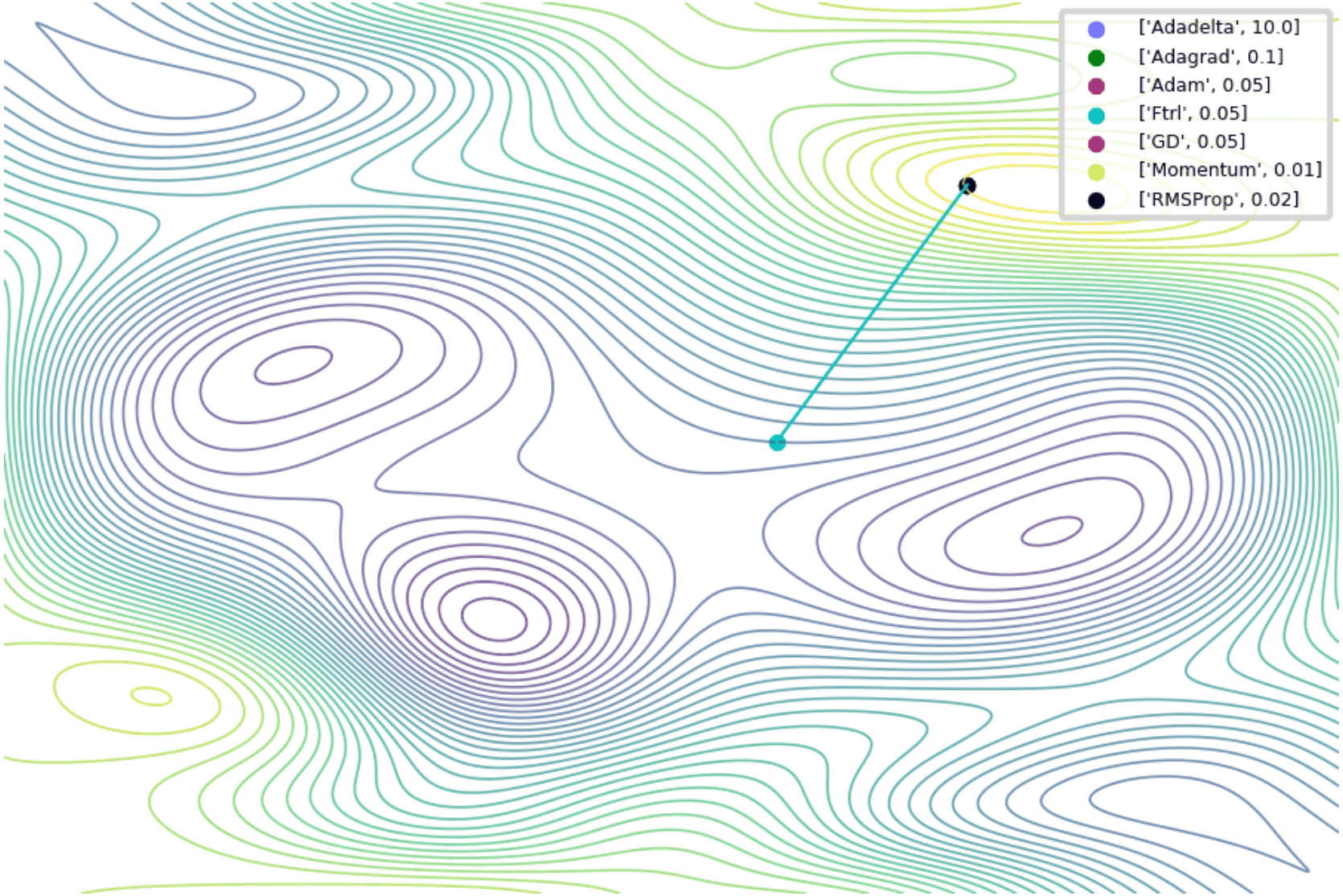
Optimization

- **SGD optimization on saddle point**



Optimization

- **Convergence**



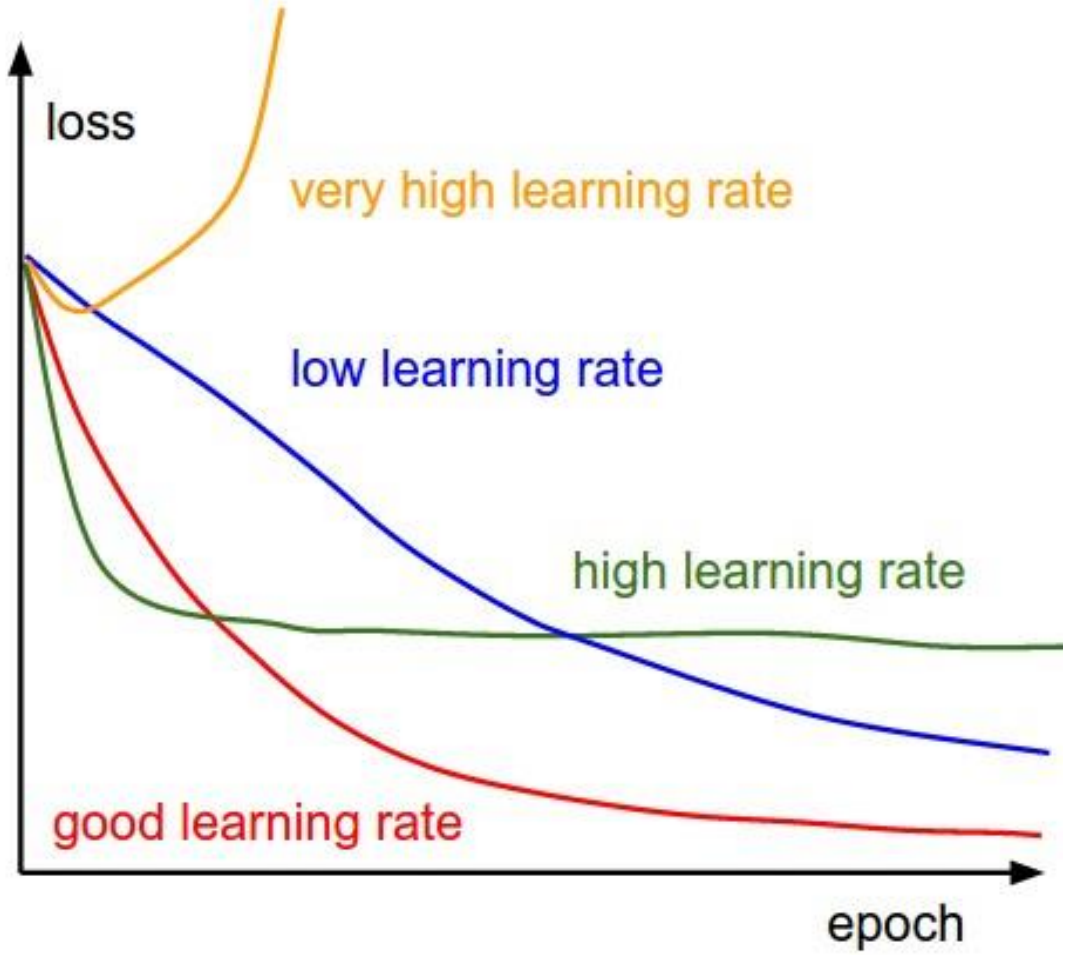
Optimization

- **Which, What, and When?**
  - Standard: **Adam**
  - Fallback option: **SGD with momentum**
  - There are few others:
    - ‘Vanilla’ SGD
    - RMSProp
    - Adagrad
    - Adadelta
    - AdaMax
    - Nada
    - AMSGrad

Learning Rate

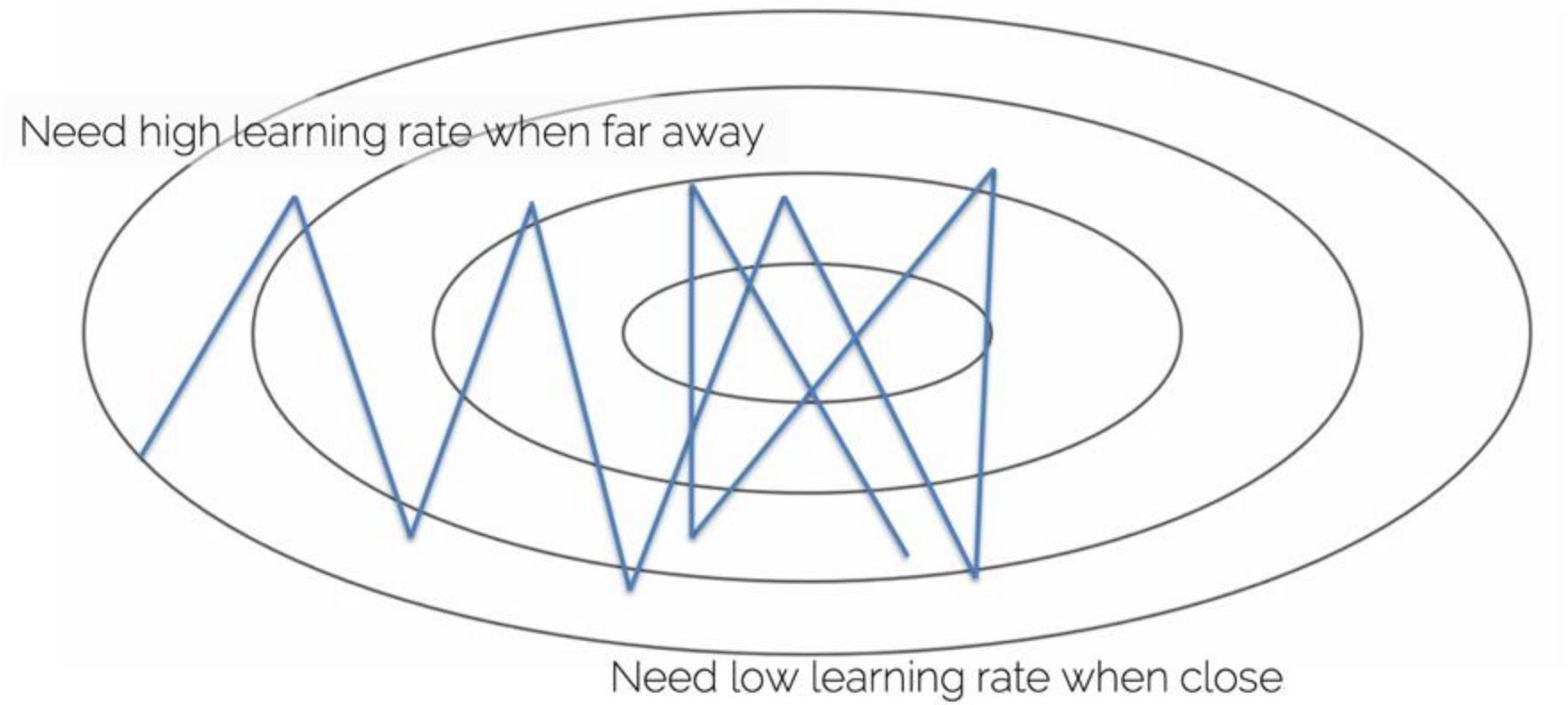
Learning Rate

- **Implications**
  - What if too high?
  - What if too low?



Learning Rate

- **Implications**
  - What if too high?
  - What if too low?



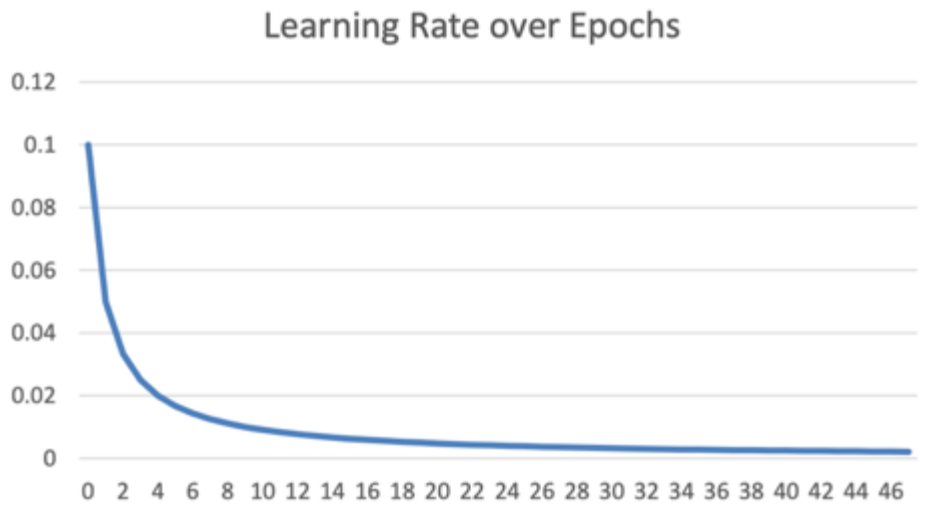
Learning Rate

- **Learning Rate Decay**

- $\alpha = \frac{1}{1+decay_rate*epoch} \cdot \alpha_0$

  - E.g., $\alpha_0 = 0.1$, $decay_rate = 1.0$

→ Epoch 0: **0.1**
→ Epoch 1: **0.05**
→ Epoch 2: **0.033**
→ Epoch 3: **0.025**
…



Learning Rate

- **Learning Rate Decay (Many Options)**
  - Step decay $\alpha = \alpha - t \cdot \alpha$ (only every n steps)
    → T is decay rate (often 0.5)
  - Exponential decay $\alpha = t^{epoch} \cdot \alpha_0$
    → t is decay rate (t < 1.0)
  - $\alpha = \frac{t}{\sqrt{epoch}} \cdot a_0$
    → t is decay rate
  - Etc.

Learning Rate

- **Training Schedule**
  - Manually specify learning rate for entire training process
  - Manually set learning rate every n-epochs → How?
    - Trial and error (the hard way)
    - Some experience (only generalizes to some degree)
  - Consider: #epochs, training set size, network size, etc.

Thank You